



دانشگاه دامغان

دانشکده فنی و مهندسی

پایان نامه دوره کارشناسی مهندسی کامپیوتر



سایت اشتراک گذاری رابط برنامه نویسی کاربردی با پایتون

سجاد رضاقلی زاده

استاد راهنما:

دکتر زهره کریمی

تیر ۱۴۰۵



بسمه تعالی

تشکر و قدردانی

از استاد راهنما و همه کسانی که در مسیر انجام این پروژه با راهنمایی، پیگیری و دلگرمی کنار من بودند، صمیمانه سپاسگزارم.

چکیده

این گزارش درباره یک سکوی فارسی^۱ برای اشتراک گذاری، جست و جو و مصرف رابط برنامه نویسی کاربردی^۲ است؛ سامانه ای که تلاش می کند تجربه کشف سرویس، خواندن مستندات، مدیریت حساب و انتشار رابط را در یک مسیر پیوسته کنار هم قرار دهد. در این پروژه، سمت سرور^۳ با Django REST Framework و سمت کاربر^۴ با React ساخته شده و فضای کلی آن به بازارچه رابط های برنامه نویسی^۵ نزدیک است. کاربر می تواند فهرست را ببیند، مستندات را بررسی کند، وارد حساب خود شود، مصرف و اشتراک را دنبال کند، رابط منتشر کند و از ابزار فراخوان^۶ و محیط طراحی جریان^۷ کمک بگیرد.

گزارش از دو زاویه به پروژه نگاه می کند: تجربه کاربر و ساختار فنی. ابتدا روشن می شود کاربر با سامانه چه کاری انجام می دهد و سپس توضیح داده می شود این رفتارها چگونه میان رابط کاربری، سمت سرور و لایه داده تقسیم شده اند. منابع فنی از مستندات رسمی Django, Python, Django, REST Framework, React, Node.js, Docker, MongoDB, OpenAPI و RapidAPI انتخاب شده اند. برای ارزیابی سمت سرور نیز دستور `python manage.py test api` اجرا شد و نتیجه آن در فصل آزمون آمده است.

کلیدواژه ها: سکوی اشتراک گذاری رابط برنامه نویسی، React, Django REST Framework، احراز هویت، OpenAPI

Platform^۱
Application Programming Interface (API)^۲
Backend^۳
Frontend^۴
API Marketplace^۵
Caller^۶
Studio^۷

فهرست مطالب

د	فهرست علائم و نشانه‌ها
ه	فهرست جدول‌ها
و	فهرست نمودارها، عکس‌ها و نقشه‌ها
۱	۱ مقدمه
۱	۱-۱ زمینه پروژه
۱	۱-۲ بیان مسئله
۲	۱-۳ اهداف
۲	۱-۴ دامنه و محدودیت‌ها
۳	۱-۵ ذی‌نفعان
۳	۱-۶ سازمان‌دهی گزارش
۳	۱-۷ جمع‌بندی فصل
۴	۲ فناوری‌ها و پیشینه
۴	۲-۱ Django, Python و Django REST Framework
۴	۲-۲ MongoDB و مدل داده
۵	۲-۳ React, TypeScript و Vite
۵	۲-۴ Node.js و ابزارهای سمت کاربر
۵	۲-۵ Docker Compose و اجرای محلی
۶	۲-۶ OpenAPI و RapidAPI
۶	۲-۷ جمع‌بندی فصل
۷	۳ تحلیل نیازمندی‌ها
۷	۳-۱ نقش‌های کاربری
۷	۳-۲ نیازمندی‌های عملکردی و غیرفعال
۹	۳-۳ موارد کاربرد سامانه
۱۰	۳-۳-۱ نمودارهای موارد کاربرد

۳-۳-۲	مورد کاربرد UC-01: کشف و جستجوی رابط	۱۱
۳-۳-۳	مورد کاربرد UC-02: بررسی جزئیات و مستندات	۱۲
۳-۳-۴	مورد کاربرد UC-03: ثبت نام، ورود و خروج	۱۳
۳-۳-۵	مورد کاربرد UC-04: مدیریت حساب، کلید و سازمان	۱۴
۳-۳-۶	مورد کاربرد UC-05: انتخاب و فعال سازی اشتراک	۱۵
۳-۳-۷	مورد کاربرد UC-06: انتشار رابط	۱۶
۳-۳-۸	مورد کاربرد UC-07: ثبت یا اصلاح امتیاز	۱۷
۳-۳-۹	مورد کاربرد UC-08: اجرای درخواست در ابزار فراخوان	۱۸
۳-۳-۱۰	مورد کاربرد UC-09: تعریف و ذخیره جریان در محیط طراحی	۱۹
۳-۳-۱۱	مورد کاربرد UC-10: ایجاد پروژه آغازین	۲۰
۳-۳-۱۲	مورد کاربرد UC-11: مشاهده پیشخوان، دسترسی و مصرف	۲۱
۳-۳-۱۳	مورد کاربرد UC-12: مدیریت داده های زنده سامانه	۲۲
۳-۳-۱۴	مورد کاربرد UC-13: تحلیل آرشیو پروژه	۲۳
۳-۳-۱۵	مورد کاربرد UC-14: استقرار پروژه فشرده	۲۴
۳-۳-۱۶	مورد کاربرد UC-15: کار با فهرست و پروژه از خط فرمان	۲۵
۳-۳-۱۷	روابط و قابلیت ردیابی موارد کاربرد	۲۶
۳-۴	ورودی ها، خروجی ها و اعتبارسنجی	۲۷
۳-۵	امنیت	۲۷
۳-۶	جمع بندی فصل	۲۸

۴	معماری و طراحی	۲۹
۴-۱	نمای کلی	۲۹
۴-۲	نمودار معماری	۲۹
۴-۳	مدل داده	۳۰
۴-۴	مرزهای خارجی	۳۱
۴-۵	جمع بندی فصل	۳۱

۵	پیاده سازی	۳۳
۵-۱	مسیرهای سمت سرور	۳۳
۵-۲	لایه مخزن	۳۳
۵-۳	تبدیل گر داده و قرارداد پاسخ	۳۴
۵-۴	سمت کاربر	۳۴
۵-۵	جریان های اصلی پیاده سازی	۳۴
۵-۶	بخش های فنی	۳۵
۵-۷	کلاینت خط فرمان و دامنه پویا	۳۶
۵-۸	تحويل تولیدی، کشف پذیری و کارایی	۳۶
۵-۹	نمونه کد	۳۷
۵-۱۰	جمع بندی فصل	۳۷

۳۸	۶	آزمون و ارزیابی
۳۸	۶-۱	راهبرد آزمون
۳۸	۶-۲	نتایج اجرا
۳۹	۶-۳	جدول آزمون‌ها
۴۰	۶-۴	محدودیت‌های آزمون
۴۱	۶-۵	جمع‌بندی فصل

۴۲	۷	نتایج و بحث
۴۲	۷-۱	ویژگی‌های پیاده‌سازی شده
۴۲	۷-۲	نمونه خروجی رابط
۴۳	۷-۳	استقرار و کیفیت نسخه زنده
۴۳	۷-۴	نقاط قوت
۴۳	۷-۵	چالش‌ها و بدهی فنی
۴۴	۷-۶	بحث
۴۴	۷-۷	جمع‌بندی فصل

۴۵	۸	نتیجه‌گیری و کارهای آینده
۴۵	۸-۱	جمع‌بندی کار انجام‌شده
۴۵	۸-۲	دستاوردهای اصلی
۴۶	۸-۳	محدودیت‌های فعلی
۴۶	۸-۴	کارهای آینده
۴۶	۸-۵	جمع‌بندی فصل

۴۸ منابع

۵۰	الف	راهنمای اجرا و شواهد تکمیلی
۵۰	۱-الف	اجرای محلی
۵۰	۲-الف	اجرای آزمون سمت سرور
۵۱	۳-الف	کنترل کامل و استقرار زنده
۵۱	۴-الف	مسیرهای مهم

فهرست علائم و نشانه‌ها

نماد	شرح
API	رابط برنامه‌نویسی کاربردی
DRF	Django REST Framework
SPA	برنامه تک‌صفحه‌ای
RTL	راست‌به‌چپ
ORM	نگاشت شیء-رابطه

فهرست جداول

۳-۱	نیازمندی‌های اصلی پروژه	۸
۳-۳	خلاصه موارد کاربرد پروژه	۹
۳-۵	ردیابی موارد کاربرد به نیازمندی‌های پروژه	۲۶
۴-۱	گروه‌های داده اصلی	۳۰
۵-۱	جریان‌های اصلی در پیاده‌سازی	۳۴
۵-۳	بخش‌های فنی و نقش آن‌ها در پروژه	۳۵
۶-۱	خلاصه آزمون‌های اجراشده	۳۹
الف-۱	مسیرهای مهم برای بررسی دستی سامانه	۵۱

فهرست نمودارها، عکس‌ها و نقشه‌ها

۸	نمای نقش‌ها و جریان‌های اصلی پروژه	۳-۱
۱۱	نمودار PlantUML مورد کاربرد UC-01	۳-۲
۱۲	نمودار PlantUML مورد کاربرد UC-02	۳-۳
۱۳	نمودار PlantUML مورد کاربرد UC-03	۳-۴
۱۴	نمودار PlantUML مورد کاربرد UC-04	۳-۵
۱۵	نمودار PlantUML مورد کاربرد UC-05	۳-۶
۱۶	نمودار PlantUML مورد کاربرد UC-06	۳-۷
۱۷	نمودار PlantUML مورد کاربرد UC-07	۳-۸
۱۸	نمودار PlantUML مورد کاربرد UC-08	۳-۹
۱۹	نمودار PlantUML مورد کاربرد UC-09	۳-۱۰
۲۰	نمودار PlantUML مورد کاربرد UC-10	۳-۱۱
۲۱	نمودار PlantUML مورد کاربرد UC-11	۳-۱۲
۲۲	نمودار PlantUML مورد کاربرد UC-12	۳-۱۳
۲۳	نمودار PlantUML مورد کاربرد UC-13	۳-۱۴
۲۴	نمودار PlantUML مورد کاربرد UC-14	۳-۱۵
۲۵	نمودار PlantUML مورد کاربرد UC-15	۳-۱۶
۳۰	معماری سطح بالای پروژه	۴-۱

فصل ۱

مقدمه

در شروع گزارش، بهتر است پروژه را از همان جایی ببینیم که کاربر با آن روبه‌رو می‌شود: محیطی فارسی برای پیدا کردن، بررسی، انتشار و مصرف رابط‌های برنامه‌نویسی. اگر این تصویر روشن باشد، توضیح‌های فنی فصل‌های بعد هم معنا پیدا می‌کنند؛ چون هر مسیر، جدول و تصمیم معماری در نهایت باید به تجربه‌ای قابل فهم برای کاربر یا توسعه‌دهنده برسد.

۱-۱ زمینه پروژه

IranAPI Hub سکویی فارسی برای معرفی، جستجو، انتشار و مصرف رابط‌های برنامه‌نویسی است. فضای پروژه به بازارچه‌های رابط نزدیک است؛ جایی که کاربر پیش از استفاده از یک سرویس، می‌تواند آن را بشناسد، مستنداتش را بخواند، طرح‌های اشتراک را مقایسه کند و پس از ورود، مصرف و دسترسی‌های خود را مدیریت کند. این نگاه با مدل رایج مرکز و بازارچه رابط‌های برنامه‌نویسی هم‌خوان است [۱۱].

از دید فنی، پروژه روی ترکیب Python, Django REST Framework, React, Vite, MongoDB و Docker Compose شکل گرفته است [۱، ۲، ۳، ۴، ۶، ۹، ۸]. این ترکیب برای سامانه‌ای مناسب است که هم به رابط برنامه‌نویسی ساخت‌یافته نیاز دارد و هم به رابط کاربری سریع، فارسی و توسعه‌پذیر.

۱-۲ بیان مسئله

کاربر چنین سامانه‌ای معمولاً دنبال فهرستی ساده از سرویس‌ها نیست. او می‌خواهد سریع بفهمد هر رابط چه کاری انجام می‌دهد، مستندات هر نقطه پایانی^۱ چگونه نوشته شده، چه طرح‌هایی دارد و پس از ورود، وضعیت مصرف، کلیدها، اشتراک و سازمان خود را کجا می‌تواند ببیند. از سوی دیگر، توسعه‌دهنده هم فقط مصرف‌کننده نیست؛ باید بتواند رابط تازه منتشر کند، درخواست نمونه بفرستد، جریان کاری خود را در محیط طراحی تنظیم کند و پروژه فشرده خود را پیش از استقرار

^۱Endpoint

تحلیل و در محیط جداشده اجرا کند.

مسئله اصلی از همین جا شروع می‌شود: تجربه کاربر نباید پخش و تکه‌تکه باشد. وقتی مستندات، پرداخت، احراز هویت، مصرف و ابزار تست هر کدام در فضای جداگانه‌ای قرار بگیرند، کاربر برای رسیدن به نتیجه چند بار مسیر ذهنی خود را عوض می‌کند. این پروژه همین اجزا را در یک تجربه منسجم جمع می‌کند؛ تجربه‌ای که برای کاربر عمومی قابل فهم باشد و برای توسعه‌دهنده هم کاربرد عملی داشته باشد.

۱-۳ اهداف

با این نگاه، هدف‌های اصلی پروژه چنین تعریف شده‌اند:

- ارائه کاتالوگ رابط‌های برنامه‌نویسی با جستجو، دسته‌بندی، امتیاز، طرح اشتراک و مستندات.
 - فراهم کردن ثبت‌نام، ورود، خروج و نشست کاربر با مدل رایج احراز هویت وب.
 - نمایش پیشخوان^۲ توسعه‌دهنده شامل نمایه، کلید دسترسی^۳، مصرف، سازمان و اشتراک.
 - پشتیبانی از انتشار رابط، اجرای درخواست نمونه در ابزار فراخوان و ساخت جریان در محیط طراحی.
 - تولید اسکلت پروژه، تحلیل ایستای آرشیو و صف‌بندی استقرار محدودشده برای پروژه‌های توسعه‌دهنده.
 - ارائه قرارداد OpenAPI برای خوانایی بهتر مسیرها و امکان اتصال ابزارهای دیگر [۱۰].
- این هدف‌ها فقط به ظاهر سامانه مربوط نیستند. کاتالوگ و مستندات بخش عمومی محصول را می‌سازند؛ ورود، اشتراک و مصرف بخش خصوصی را شکل می‌دهند؛ و OpenAPI کمک می‌کند قرارداد سرویس هم برای انسان خوانا باشد و هم برای ابزارهای بیرونی قابل استفاده بماند.

۱-۴ دامنه و محدودیت‌ها

دامنه گزارش همان نسخه فعلی پروژه است: برنامه وب فارسی، رابط برنامه‌نویسی سمت سرور، اجرای محلی و جریان‌های اصلی کاربر. اجرای محلی با Docker Compose هم در همین دامنه قرار می‌گیرد، چون برای راه‌اندازی چند سرویس کنار هم طراحی شده است [۸].

در کنار قابلیت‌ها، چند محدودیت هم باید صریح گفته شود. تنظیمات تولیدی هنوز کامل نیست، بخشی از رفتارها برای محیط توسعه مناسب‌ترند و پرداخت واقعی و اتصال دارای اطلاعات اعتبارسنجی^۴ به سرویس‌های بیرونی نیاز به سخت‌سازی دارد. آزمون مرورگر اجرا شده، اما باید در چرخه یکپارچه‌سازی پیوسته نیز منظم شود. بنابراین گزارش، پروژه را به عنوان نمونه‌ای کامل و توسعه‌پذیر بررسی می‌کند، نه محصولی که همین حالا برای بهره‌برداری عمومی آماده باشد.

^۲Dashboard

^۳API Key

^۴Credential

۱-۵ ذی‌نفعان

ذی‌نفعان اصلی پروژه شامل کاربر مهمان، توسعه‌دهنده احراز شده، مدیر سیستم، ارائه‌دهنده ورود اجتماعی، سرویس مقصد رابط و بازارهای بیرونی مانند RapidAPI هستند [۱۱]. انتظار این گروه‌ها یکسان نیست: مهمان به شناخت سریع سرویس نیاز دارد، توسعه‌دهنده به ابزار و داده خصوصی، مدیر به کنترل سامانه، و سرویس‌های بیرونی به قراردادی قابل اعتماد.

۱-۶ سازمان‌دهی گزارش

ادامه گزارش به همین ترتیب پیش می‌رود: ابتدا فناوری‌های استفاده‌شده معرفی می‌شوند، سپس نیازمندی‌ها و معماری بررسی می‌شوند. بعد از آن، پیاده‌سازی، آزمون، نتایج و در پایان مسیر ادامه کار توضیح داده می‌شود.

۱-۷ جمع‌بندی فصل

در این فصل روشن شد که پروژه فقط تمرینی برای سمت سرور یا رابط کاربری ساده نیست؛ سکویی برای اشتراک‌گذاری و مصرف رابط‌های برنامه‌نویسی است. به همین دلیل، گزارش باید هم تجربه کاربر را جدی بگیرد و هم قرارداد فنی میان سمت کاربر، سمت سرور و داده را دقیق توضیح دهد.

فصل ۲

فناوری‌ها و پیشینه

انتخاب فناوری در این پروژه فقط بر اساس رایج بودن ابزارها انجام نشده است. هر بخش باید با نقش خود در محصول هماهنگ باشد: سمت سرور باید قرارداد رابط برنامه‌نویسی را روشن نگه دارد، سمت کاربر باید تجربه‌ای فارسی و سریع بسازد و اجرای محلی نباید برای ارائه و آزمون دردسر اضافه ایجاد کند. بنابراین معرفی فناوری‌ها در ادامه، بیشتر بر نقش آن‌ها در خود محصول تمرکز دارد.

۲-۱ Python، Django و Django REST Framework

Python زبان اصلی سمت سرور است؛ زبانی خوانا و پرکاربرد که برای توسعه وب و ساخت سرویس‌های داده‌محور انتخاب مناسبی به شمار می‌آید [۱]. روی این زبان، Django چارچوب اصلی وب را فراهم می‌کند و بخش‌هایی مانند تنظیمات، مسیرها، امنیت پایه، مدیریت کاربران، پنل مدیریت و ابزارهای آزمون را در اختیار پروژه می‌گذارد [۲]. Django REST Framework نقش تکمیل‌کننده همین ساختار را دارد. مفاهیمی مانند نما^۱، تبدیل‌گر داده^۲، اعتبارسنجی ورودی و پاسخ JSON کمک می‌کنند مسیرهای عمومی و خصوصی سامانه با قرارداد منظم‌تری نوشته شوند [۳]. در این پروژه، همین ترکیب پایه ساخت کاتالوگ، حساب کاربری، اشتراک، مصرف و ابزارهای توسعه‌دهنده است.

۲-۲ MongoDB و مدل داده

داده‌های این پروژه شکل کاملاً یکسانی ندارند. اطلاعات کاربر، نشست، رابط برنامه‌نویسی، طرح اشتراک، مصرف و جریان طراحی هر کدام جزئیات خود را دارند. به همین دلیل، مدل سندی برای این فضا طبیعی‌تر است. MongoDB نیز در مستندات رسمی خود همین رویکرد سندمحور را برای ذخیره داده‌های انعطاف‌پذیر توضیح می‌دهد [۹]. در گزارش، وقتی از لایه داده صحبت می‌شود،

^۱View

^۲Serializer

منظور بخشی است که همین داده‌های محصول را برای مسیرهای سمت سرور آماده می‌کند. این انتخاب برای بازارچه رابط‌های برنامه‌نویسی منطقی است. مستندات یک سرویس، طرح‌های قیمت‌گذاری، رخدادهای مصرف، جریان‌های طراحی و تحلیل آرشیو پروژه همیشه مانند رکوردهای ساده و ثابت رفتار نمی‌کنند. مدل سندی باعث می‌شود این تفاوت‌ها با انعطاف بیشتری نگهداری شوند.

۲-۳ Vite و TypeScript و React

سمت کاربر با React ساخته شده است؛ کتابخانه‌ای که برای ساخت رابط‌های کاربری مؤلفه‌محور^۳ استفاده می‌شود [۴]. برای پروژه‌ای که صفحه‌های متنوعی مانند کاتالوگ، جزئیات سرویس، پیشخوان و محیط طراحی جریان دارد، این مدل کمک می‌کند اجزای تکرارشونده جدا و نگهداری‌پذیر بمانند. TypeScript هم به کد سمت کاربر نوع‌دهی اضافه می‌کند تا خطاهای ساده زودتر دیده شوند و قرارداد بین بخش‌های مختلف روشن‌تر بماند [۵].

Vite ابزار توسعه و ساخت^۴ سمت کاربر است. مستندات رسمی Vite بر سرعت راه‌اندازی و تجربه توسعه سریع تأکید دارد [۶]. این ویژگی در کار روزمره پروژه مهم است، چون تغییر در صفحه‌ها و مؤلفه‌ها باید سریع دیده شود و توسعه رابط کاربری کند نشود.

۲-۴ Node.js و ابزارهای سمت کاربر

زیست‌بوم سمت کاربر پروژه بر Node.js و ابزارهای بسته‌بندی جاوااسکریپت استوار است. Node.js اجرای جاوااسکریپت خارج از مرورگر را فراهم می‌کند و پایه بسیاری از ابزارهای توسعه سمت کاربر است [۷]. کتابخانه‌هایی مانند TanStack Query، Axios و مجموعه‌های مؤلفه‌ای، ارتباط با سمت سرور، مدیریت وضعیت درخواست‌ها و ساخت فرم‌ها و جدول‌ها را ساده‌تر می‌کنند.

در گزارش، نام این ابزارها بدون ترجمه آمده است؛ چون برای بسیاری از آن‌ها معادل فارسی دقیق و جاافتاده‌ای وجود ندارد و ترجمه اجباری، متن را کم‌دقت‌تر می‌کند.

همین محیط Node.js برای کارخواه خط فرمان^۵ نیز به کار رفته است. این کارخواه با ذخیره امن نشانه دسترسی، خروجی جدولی یا JSON و گزینه انتظار برای استقرار، قرارداد نسخه‌دار سامانه را برای برنامه‌های خودکار قابل استفاده می‌کند.

۲-۵ Docker Compose و اجرای محلی

Docker Compose برای تعریف و اجرای چند سرویس کنار هم استفاده می‌شود [۸]. در این پروژه، همین ابزار باعث می‌شود بک‌اند و فرانت‌اند کنار هم بالا بیایند و مسیر ارتباطی مشخصی داشته

Component-based^۳
Build^۴
Command-line Client^۵

باشند. این روش برای ارائه، آزمون دستی و توسعه محلی مناسب است، هرچند جای پیکربندی تولیدی را نمی‌گیرد.

در مسیر استقرار پروژه، Docker نقش جداگانه‌ای دارد: آرشیو پس از تحلیل ایستا در کارگر صف استخراج می‌شود، تصویر ساخته می‌شود و محفظه اجرایی^۶ با محدودیت حافظه، پردازنده، فرایند و قابلیت‌ها اجرا می‌گردد.

۲-۶ RapidAPI و OpenAPI

OpenAPI برای توصیف قرارداد رابط برنامه‌نویسی به کار می‌رود؛ یعنی مسیرها، ورودی‌ها، خروجی‌ها و پاسخ‌ها را به شکلی می‌نویسد که هم انسان و هم ابزار بتوانند آن را بخوانند [۱۰]. RapidAPI نیز نمونه شناخته‌شده‌ای از فضای بازارچه رابط برنامه‌نویسی است و برای فهم تجربه کاربر در کشف، مقایسه و مصرف رابط مرجع خوبی به شمار می‌آید [۱۱].

۲-۷ جمع‌بندی فصل

پشته فنی پروژه با ماهیت آن هماهنگ است: Django REST Framework برای ساخت رابط، React برای تجربه کاربری، MongoDB برای داده‌های سندی، Docker Compose برای اجرای محلی، و OpenAPI برای خوانایی قرارداد سرویس. نتیجه این ترکیب، گزارشی است که فقط درباره ابزارها حرف نمی‌زند؛ بلکه نشان می‌دهد هر ابزار چه نقشی در شکل‌گیری محصول دارد.

فصل ۳

تحلیل نیازمندی‌ها

نیازمندی‌های این پروژه از دل تجربه کاربر بیرون می‌آیند. کاربر مهمان می‌خواهد سرویس‌ها را راحت پیدا کند؛ توسعه‌دهنده می‌خواهد بعد از ورود، حساب، مصرف و ابزارهای خود را مدیریت کند؛ مدیر هم باید بتواند سامانه را کنترل و نگهداری کند. بنابراین نیازمندی‌ها فقط یک فهرست اداری نیستند، بلکه توضیح می‌دهند هر بخش از سامانه باید چه مشکلی را حل کند.

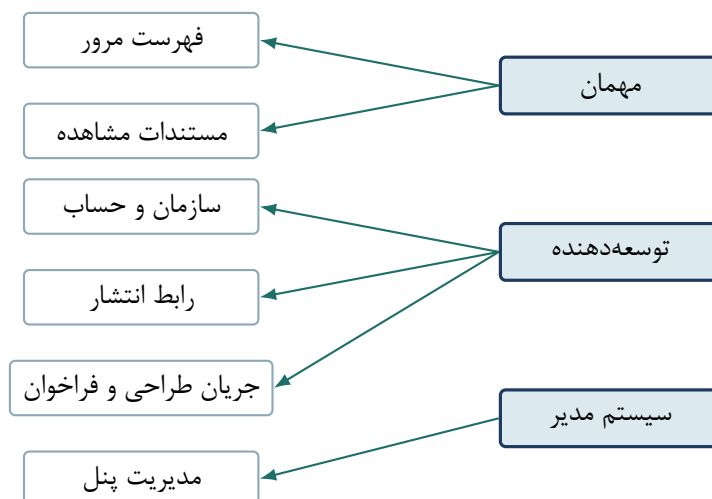
۳-۱ نقش‌های کاربری

سه نقش اصلی در پروژه دیده می‌شود: مهمان، توسعه‌دهنده احراز شده و مدیر سیستم. مهمان با بخش عمومی فهرست سروکار دارد. توسعه‌دهنده پس از ورود، وارد فضای کاری خود می‌شود و با پیشخوان، انتشار رابط، ابزار فراخوان و محیط طراحی جریان کار می‌کند. مدیر سیستم هم مسئول کنترل کلی و نگهداری داده‌هاست.

مهمان بدون ورود می‌تواند بخش عمومی بازارچه را ببیند؛ این بخش برای معرفی رابط‌ها و ایجاد اعتماد اولیه ضروری است. توسعه‌دهنده پس از احراز هویت به امکاناتی می‌رسد که با داده خصوصی او ارتباط دارند، مانند کلید دسترسی، سازمان، اشتراک، مصرف، انتشار و ابزارهای آزمایشی. نقش مدیر سیستم بیشتر به نگهداری داده و پشتیبانی از مسیرهای مدیریتی مربوط می‌شود و به همین دلیل از نظر سطح دسترسی حساس‌تر است.

۳-۲ نیازمندی‌های عملکردی و غیرفعال

جدول ۳-۱ نیازمندی‌ها را در دو گروه اصلی نشان می‌دهد. نیازمندی‌های عملکردی همان کارهایی هستند که کاربر مستقیم با آن‌ها روبه‌رو می‌شود؛ مانند جستجوی رابط، ورود، انتشار یا مشاهده مصرف. نیازمندی‌های امنیتی و اجرایی کیفیت همین رفتارها را حفظ می‌کنند؛ مانند پنهان کردن کلیدها، پاسخ خطای قابل فهم و اجرای محلی با Docker Compose.



شکل ۳-۱ نمای نقش ها و جریان های اصلی پروژه

جدول ۳-۱ نیازمندی های اصلی پروژه

شناسه	نیازمندی	نوع	اولویت	وضعیت
R1	نمایش فهرست رابط ها با صفحه بندی، جستجو، دسته بندی و مرتب سازی	عملکردی	زیاد	انجام شده
R2	نمایش جزئیات، طرح ها، مستندات، نقاط پایانی و رابط های مشابه	عملکردی	زیاد	انجام شده
R3	ثبت نام، ورود، نشست، خروج و کشف ارائه دهندگان ورود اجتماعی	عملکردی	زیاد	انجام شده
R4	مدیریت کاربر، نمایه، سازمان، اشتراک، پرداخت دستی و مصرف	عملکردی	زیاد	انجام شده
R5	انتشار رابط توسط کاربر احراز شده	عملکردی	متوسط	انجام شده
R6	اجرای درخواست نمونه در ابزار فراخوان و ثبت مصرف	عملکردی	متوسط	انجام شده
R7	اعتبارسنجی و ذخیره تعریف جریان طراحی همراه ثبت رویداد مصرف، بدون اجرای گره ها	عملکردی	متوسط	انجام شده
R8	بازگشت خطای یکنواخت برای اعتبارسنجی و دسترسی	غیرفعال	زیاد	انجام شده

شناسه	نیازمندی	نوع	اولویت	وضعیت
R9	پنهان سازی کلیدها و توکن ها در پاسخ ها و لاگ ها	امنیتی	زیاد	انجام شده
R10	اجرای محلی دو سرویس با استقرار Docker Compose	استقرار	متوسط	مستند
R11	تحلیل ایمن آرشیو پروژه فشرده بدون اجرای کد	امنیتی	زیاد	انجام شده
R12	صف بندی و استقرار ایزوله پروژه در محیط محدود شده	استقرار	متوسط	انجام شده
R13	مصرف قرارداد نسخه دار از کلاینت خط فرمان عملکردی	متوسط	انجام شده	

اولویت زیاد برای نیازهایی در نظر گرفته شده که نبود آنها هسته محصول را مختل می کند؛ مانند فهرست، احراز هویت، حساب کاربری و امنیت داده حساس. اولویت متوسط برای قابلیت هایی است که ارزش محصول را کامل تر می کنند، اما می توانند در نسخه های بعدی هم رشد کنند؛ مانند محیط طراحی جریان، ابزار فراخوان و اجرای محلی.

۳-۳ موارد کاربرد سامانه

مورد کاربرد، رفتار سامانه را از دید بازیگر و برای رسیدن به یک هدف مشخص توصیف می کند. در این بخش، هر مورد کاربرد علاوه بر نام و بازیگر، با پیش شرط، محرک، جریان اصلی، مسیرهای جایگزین و پس شرط بیان می شود. این قالب کمک می کند مرز میان قابلیت پیاده سازی شده، خطای قابل انتظار و نتیجه نهایی هر تعامل روشن باشد.

جدول ۳-۳ نمای کلی موارد کاربرد را نشان می دهد. مهمان به مسیرهای عمومی دسترسی دارد؛ توسعه دهنده احراز شده علاوه بر آنها می تواند داده و ابزارهای اختصاصی خود را مدیریت کند؛ مدیر سیستم نیز از مسیر مدیریتی جداگانه برای نگهداری داده های سامانه استفاده می کند.

جدول ۳-۳ خلاصه موارد کاربرد پروژه

شناسه	عنوان	بازیگر اصلی	نتیجه مورد انتظار
UC-01	کشف و جستجوی رابط	مهمان یا توسعه دهنده	یافتن سرویس متناسب با نیاز
UC-02	بررسی جزئیات و مستندات	مهمان یا توسعه دهنده	شناخت قرارداد و شرایط مصرف سرویس
UC-03	ثبت نام، ورود و خروج	مهمان برای ورود؛ کاربر احراز شده برای خروج	ایجاد نشست امن یا پایان دادن به آن
UC-04	مدیریت حساب و سازمان	توسعه دهنده	به روز رسانی هویت، کلید و سازمان ها

شناسه	عنوان	بازیگر اصلی	نتیجه مورد انتظار
UC-05	انتخاب و فعال سازی اشتراک	توسعه دهنده	ثبت یا لغو فرایند خرید و فعال شدن طرح
UC-06	انتشار رابط	توسعه دهنده ارائه دهنده	افزوده شدن سرویس به فهرست عمومی
UC-07	ثبت امتیاز برای رابط	توسعه دهنده مصرف کننده	ثبت یا اصلاح نظر عددی کاربر
UC-08	اجرای درخواست در ابزار فراخوان	مهمان یا توسعه دهنده	دریافت پاسخ واقعی و، در حالت خصوصی، ثبت مصرف
UC-09	تعریف و ذخیره جریان در محیط طراحی	توسعه دهنده	ذخیره تعریف جریان و ثبت رویداد مصرف، بدون اجرای گره ها
UC-10	ایجاد پروژه آغازین	توسعه دهنده	تولید ساختار اولیه پروژه برای مصرف رابط
UC-11	مشاهده پیشخوان و مصرف	توسعه دهنده	پایش دسترسی ها، اشتراک و سابقه استفاده
UC-12	مدیریت داده های زنده سامانه	مدیر سیستم توسعه دهنده رابط با دسترسی محدود	مدیریت مجموعه های مجاز با کنترل نقش و مالکیت
UC-13	تحلیل آرشیو پروژه	توسعه دهنده	تشخیص زبان، چارچوب، مسیرها و آمادگی استقرار
UC-14	استقرار پروژه فشرده	توسعه دهنده	ساخت و اجرای ایزوله با نشانی قابل بررسی
UC-15	کار با فهرست و پروژه از خط فرمان	کاربر توسعه دهنده برای فرمان های خصوصی	خودکارسازی بررسی و مصرف سرویس

۱-۳-۳ نمودارهای موارد کاربرد

برای جلوگیری از ابهام در نقش ها و مرز سامانه، هر مورد کاربرد با یک نمودار مستقل زبان مدل سازی پلنتیو ام ال^۱ در همان صفحه مشخصات نمایش داده شده است. بازیگر اصلی و بازیگران بیرونی خارج از مرز IranAPI قرار دارند و رابطه های «شامل»^۲ و «گسترش»^۳ فقط رفتارهای واقعاً پیاده سازی شده را نشان می دهند. فایل های منبع نمودارها همراه پروژه نگهداری می شوند تا نمودار و پیاده سازی همزمان قابل اصلاح باشند.

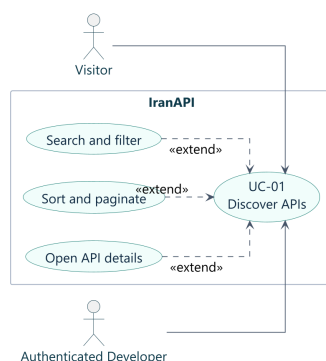
^۱ PlantUML

^۲ include

^۳ extend

۳-۳-۲ مورد کاربرد UC-01: کشف و جستجوی رابط

بازیگر اصلی: مهمان یا توسعه‌دهنده احراز شده.
هدف: یافتن یک رابط فعال بر اساس مسئله، حوزه کاربرد یا محبوبیت.
پیش شرط: فهرست در دسترس باشد؛ ورود به حساب لازم نیست.
محرک: کاربر صفحه مرور سرویس‌ها را باز می‌کند یا عبارت جست‌وجو و پالایه را تغییر می‌دهد.



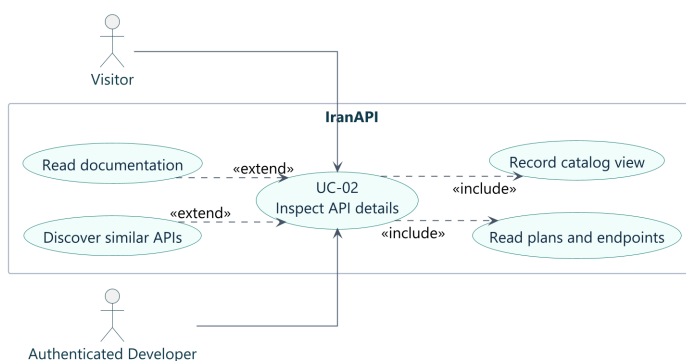
شکل ۳-۲ نمودار PlantUML مورد کاربرد UC-01

جریان اصلی:

۱. سامانه فهرست دسته‌ها و رابط‌های فعال را از سمت سرور دریافت می‌کند.
 ۲. کاربر می‌تواند عبارت جست‌جو، دسته، برچسب، وضعیت ویژه یا محبوب و ترتیب نمایش را مشخص کند.
 ۳. سمت سرور شرط‌ها را روی فهرست اعمال و نتیجه را صفحه‌بندی می‌کند.
 ۴. رابط کاربری برای هر نتیجه، اطلاعات خلاصه مانند نام، توضیح کوتاه، دسته، امتیاز و قیمت پایه را نشان می‌دهد.
 ۵. کاربر یکی از نتایج را برای مطالعه بیشتر انتخاب می‌کند.
- مسیرهای جایگزین و خطا:** اگر هیچ سرویس منطبق نباشد، سامانه حالت خالی را نشان می‌دهد و کاربر می‌تواند پالایه‌ها را حذف کند. سرویس غیرفعال در نتیجه عمومی دیده نمی‌شود. درخواست مشاهده سرویس‌های متعلق به کاربر نیز فقط پس از احراز هویت پذیرفته می‌شود.
- پس شرط:** فهرست منطبق با معیارهای کاربر نمایش داده می‌شود؛ تا زمانی که جزئیات یک سرویس باز نشود، داده عملیاتی کاربر تغییر نمی‌کند.

۳-۳-۳ مورد کاربرد UC-02: بررسی جزئیات و مستندات

بازیگر اصلی: مهمان یا توسعه‌دهنده احراز شده.
هدف: ارزیابی فنی و اقتصادی سرویس پیش از استفاده.
پیش شرط: رابط انتخاب شده فعال و دارای شناسه متنی معتبر باشد.
محرک: کاربر کارت یک سرویس یا پیوند مستقیم صفحه جزئیات را باز می‌کند.



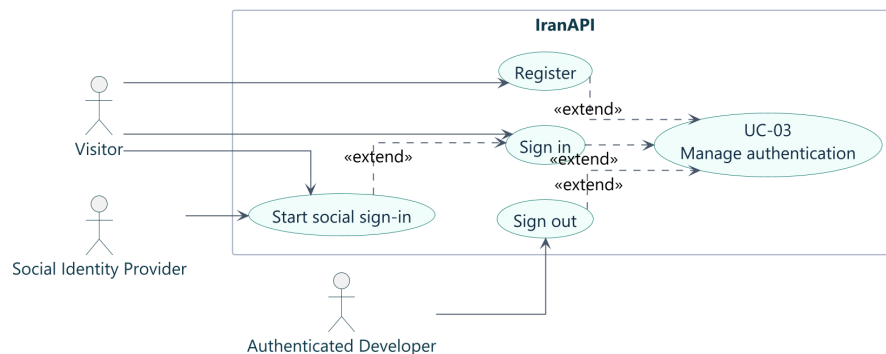
شکل ۳-۳ نمودار PlantUML مورد کاربرد UC-02

جریان اصلی:

۱. سامانه اطلاعات کامل سرویس را با دسته، توضیحات، نشانی پایه و روش احراز هویت دریافت می‌کند.
 ۲. طرح‌های قیمت‌گذاری، مستندات و نقاط پایانی ثبت شده کنار اطلاعات اصلی نمایش داده می‌شوند.
 ۳. کاربر روش، مسیر، ورودی و توضیح هر نقطه پایانی را بررسی می‌کند.
 ۴. سامانه سرویس‌های مشابه را بر اساس داده فهرست پیشنهاد می‌دهد.
 ۵. شمار مشاهده سرویس برای سنجش میزان توجه کاربران افزایش می‌یابد.
- مسیرهای جایگزین و خطا:** اگر سرویس وجود نداشته یا غیرفعال باشد، پاسخ «یافت نشد» برگردانده می‌شود. نبود طرح، مستندات یا نقطه پایانی مانع نمایش اطلاعات پایه نیست و رابط کاربری برای آن بخش حالت خالی نشان می‌دهد.
- پس شرط:** کاربر اطلاعات لازم برای مقایسه، انتخاب طرح، آزمایش یا کنارگذاشتن سرویس را در اختیار دارد.

۳-۳-۴ مورد کاربرد UC-03: ثبت نام، ورود و خروج

بازیگران اصلی: مهمان برای ثبت نام و ورود؛ توسعه دهنده احراز شده برای خروج.
هدف: ایجاد حساب جدید، بازیابی نشست حساب موجود یا پایان امن نشست.
پیش شرط: برای ثبت نام، نام کاربری و رایانامه باید تکراری نباشند؛ برای ورود، حساب باید از قبل وجود داشته باشد.
محرک: کاربر فرم ثبت نام یا ورود را ارسال می کند، یا فرمان خروج را می زند.



شکل ۳-۴ نمودار PlantUML مورد کاربرد UC-03

جریان اصلی ثبت نام و ورود:

۱. کاربر مشخصات هویتی و گذرواژه را وارد می کند.
۲. سامانه کامل بودن داده، تطابق گذرواژه ها و یکتایی مشخصات را اعتبارسنجی می کند.
۳. در ثبت نام، سند کاربر و نمایه ساخته می شود؛ در ورود، گذرواژه با داده ذخیره شده تطبیق داده می شود.
۴. سمت سرور یک نشست پایدار می سازد و شناسه آن را با کوکی HttpOnly به مرورگر می دهد.
۵. اطلاعات عمومی حساب و نمایه برای راه اندازی فضای خصوصی به رابط کاربری برگردانده می شود.

جریان خروج: سامانه نشست سمت سرور را حذف، کوکی مرورگر را منقضی و وضعیت کاربر را مهمان می کند.

مسیرهای جایگزین و خطا: نام کاربری یا رایانامه تکراری، عدم تطابق گذرواژه ها و اطلاعات ورود نادرست با خطای اعتبارسنجی پاسخ داده می شوند. ارائه دهنده ورود اجتماعی فقط در صورت پیکربندی و فعال بودن قابل استفاده است؛ ارائه دهنده ناشناخته یا غیرفعال کاربر را وارد سامانه نمی کند.

پس شرط: در موفقیت، نشست احراز شده فعال است؛ پس از خروج، مسیرهای خصوصی دوباره نیازمند ورود هستند.

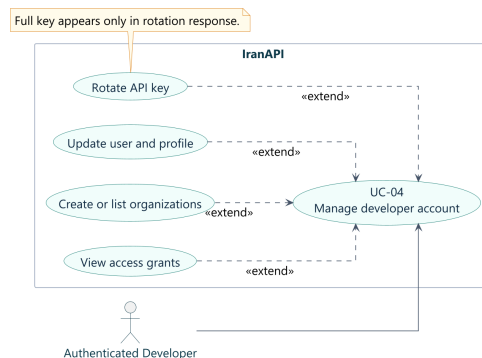
۳-۳-۵ مورد کاربرد UC-04: مدیریت حساب، کلید و سازمان

بازیگر اصلی: توسعه‌دهنده احراز شده.

هدف: نگهداری اطلاعات حساب و منابع هویتی مرتبط با فعالیت توسعه‌دهنده.

پیش شرط: نشست یا توکن معتبر وجود داشته باشد.

محرک: کاربر صفحه حساب یا سازمان‌ها را باز می‌کند یا درخواست تغییر می‌فرستد.



شکل ۳-۵ نمودار PlantUML مورد کاربرد UC-04

جریان اصلی:

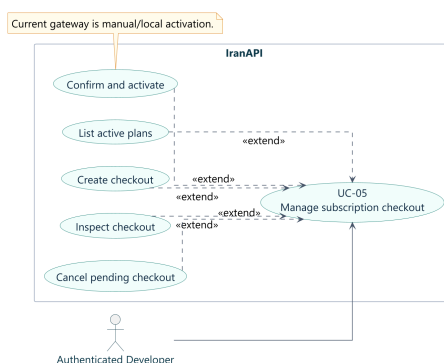
۱. سامانه اطلاعات کاربر و نمایه را با حذف یا پوشاندن مقادیر حساس نمایش می‌دهد.
۲. کاربر مشخصات مجاز مانند نام، نام خانوادگی یا رایانامه را اصلاح می‌کند.
۳. در صورت نیاز، کاربر کلید دسترسی جدید می‌سازد؛ مقدار کامل کلید فقط در پاسخ همان عملیات نمایش داده می‌شود.
۴. در مشاهده‌های بعدی فقط پیش‌نمایش پوشانده شده کلید برمی‌گردد.
۵. کاربر سازمانی با نام و ناحیه ایجاد می‌کند یا فهرست سازمان‌های متعلق به خود را می‌بیند.
۶. سامانه مجوزهای دسترسی فعال کاربر به رابط‌ها و طرح‌های مربوط را نیز قابل مشاهده می‌کند.

مسیرهای جایگزین و خطا: رایانامه تکراری یا داده نامعتبر ذخیره نمی‌شود. درخواست بدون احراز هویت رد می‌شود. روش قدیمی تولید کلید غیرفعال است و چرخش کلید باید از مسیر امن جدید انجام شود.

پس شرط: نمایه، کلید یا سازمان در مالکیت همان کاربر به‌روزرسانی می‌شود و راز کامل در پاسخ‌های عادی افشا نمی‌شود.

۳-۳-۶ مورد کاربرد UC-05: انتخاب و فعال‌سازی اشتراک

بازیگر اصلی: توسعه‌دهنده احراز شده.
هدف: انتخاب طرح اشتراک و مدیریت فرایند پرداخت دستی شبیه‌سازی شده.
پیش شرط: طرح انتخاب شده فعال باشد و کاربر نشست معتبر داشته باشد.
محرک: کاربر در صفحه قیمت‌گذاری یک طرح را انتخاب می‌کند.



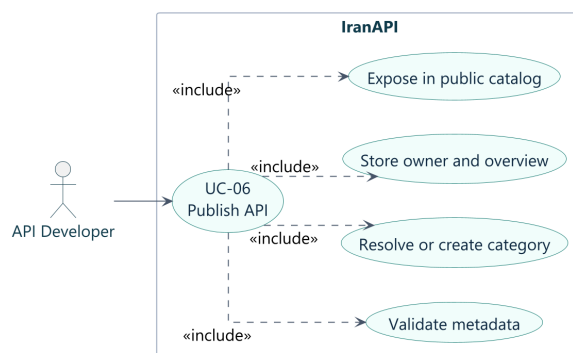
شکل ۳-۶ نمودار PlantUML مورد کاربرد UC-05

جریان اصلی:

۱. سامانه فهرست طرح‌های قابل خرید را نمایش می‌دهد.
 ۲. کاربر طرح را انتخاب و یک رکورد فرایند خرید ایجاد می‌کند.
 ۳. جزئیات مبلغ، وضعیت و زمان اعتبار فرایند خرید نمایش داده می‌شود.
 ۴. کاربر تأیید را انجام می‌دهد.
 ۵. سامانه فرایند خرید را تکمیل و اشتراک فعال را برای کاربر ثبت می‌کند.
 ۶. صفحه حساب، طرح و وضعیت اشتراک جاری را نمایش می‌دهد.
- مسیرهای جایگزین و خطا:** کاربر می‌تواند پیش از تکمیل، فرایند خرید را لغو کند. طرح یا فرایند خرید ناموجود، منقضی یا متعلق به کاربر دیگر پذیرفته نمی‌شود. تأیید دوباره عملیات تکمیل شده، اشتراک تکراری ایجاد نمی‌کند.
- پس شرط:** فرایند خرید لغو شده باقی می‌ماند یا اشتراک انتخابی به شکل فعال در حساب ثبت می‌شود.

۷-۳-۳ مورد کاربرد UC-06: انتشار رابط

بازیگر اصلی: توسعه‌دهنده ارائه‌دهنده سرویس.
هدف: معرفی یک سرویس تازه و قابل کشف کردن آن در فهرست.
پیش‌شرط: کاربر وارد حساب شده و اطلاعات فنی سرویس را در اختیار داشته باشد.
محرک: کاربر فرم انتشار را تکمیل و ارسال می‌کند.



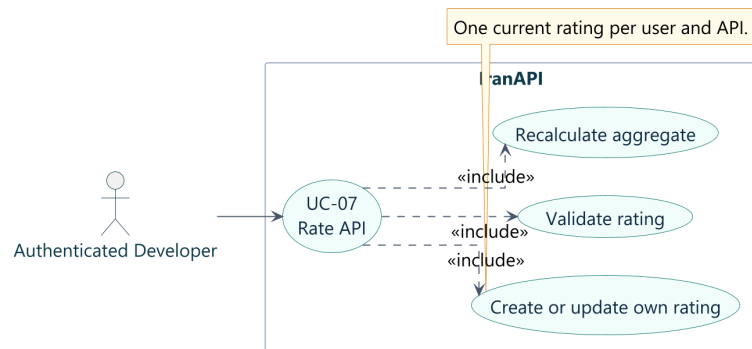
شکل ۷-۳ نمودار PlantUML مورد کاربرد UC-06

جریان اصلی:

۱. کاربر نام، توضیح، نشانی پایه، نشانی مستندات، دسته، روش احراز هویت و برچسب‌ها را وارد می‌کند.
 ۲. سامانه قالب داده و فیلدهای الزامی را اعتبارسنجی و برچسب‌ها و روش احراز هویت را یکدست می‌کند.
 ۳. اگر دسته وارد شده وجود نداشته باشد، دسته‌ای با شناسه متنی مناسب ساخته می‌شود.
 ۴. سند رابط با شناسه یکتا، مالک فعلی و وضعیت منتشرشده ایجاد می‌شود.
 ۵. توضیح یا پیوند مستندات به عنوان مدرک آغازین سرویس ثبت می‌شود.
 ۶. سرویس تازه در فهرست عمومی و فهرست سرویس‌های متعلق به کاربر قرار می‌گیرد.
- مسیرهای جایگزین و خطا:** درخواست مهمان رد می‌شود. نبود فیلد الزامی، نشانی نامعتبر یا روش احراز هویت غیرمجاز باعث خطای اعتبارسنجی می‌شود و هیچ سرویس ناقصی منتشر نمی‌گردد.
- پس‌شرط:** یک رابط فعال و قابل جست‌وجو با رابطه مالکیت مشخص در فهرست ذخیره شده است.

۳-۳-۸ مورد کاربرد UC-07: ثبت یا اصلاح امتیاز

بازیگر اصلی: توسعه‌دهنده احراز شده.
هدف: ثبت ارزیابی عددی از کیفیت یک سرویس و کمک به تصمیم کاربران دیگر.
پیش شرط: کاربر وارد شده و رابط مورد نظر فعال باشد.
محرک: کاربر در صفحه جزئیات، مقدار امتیاز را ارسال می‌کند.



شکل ۳-۸ نمودار PlantUML مورد کاربرد UC-07

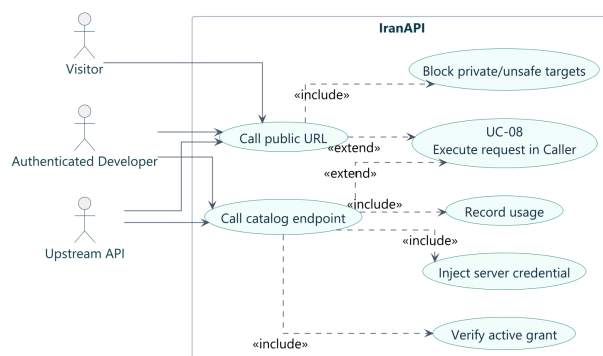
جریان اصلی:

۱. سامانه معتبر بودن دامنه امتیاز را بررسی می‌کند.
 ۲. اگر کاربر قبلاً به این سرویس امتیاز نداده باشد، رکورد تازه ایجاد می‌شود.
 ۳. اگر امتیاز قبلی وجود داشته باشد، همان رکورد به‌روزرسانی می‌شود.
 ۴. میانگین و تعداد امتیازها دوباره محاسبه و همراه امتیاز خود کاربر برگردانده می‌شود.
- مسیرهای جایگزین و خطا:** امتیاز خارج از دامنه، سرویس ناموجود یا درخواست مهمان پذیرفته نمی‌شود. هر کاربر برای هر سرویس فقط یک امتیاز جاری دارد.
- پس شرط:** امتیاز کاربر و شاخص جمعی سرویس به‌روز شده‌اند.

۳-۳-۹ مورد کاربرد UC-08: اجرای درخواست در ابزار فراخوان

بازیگر اصلی: مهمان برای درخواست مستقیم عمومی؛ توسعه‌دهنده دارای مجوز برای نقطه پایانی فهرست.

هدف: آزمایش یک درخواست واقعی و مشاهده کد وضعیت، زمان پاسخ، نوع محتوا و بدنه پاسخ.
پیش شرط: در حالت فهرست، کاربر باید نشست معتبر و مجوز دسترسی فعال به سرویس داشته باشد؛ تنظیمات ارائه‌دهنده و نقطه پایانی نیز باید در سرور ثبت شده باشند.
محرک: کاربر روش، مقصد، پارامترها و در صورت نیاز بدنه را در ابزار فراخوان وارد و درخواست را اجرا می‌کند.



شکل ۳-۹ نمودار PlantUML مورد کاربرد UC-08

جریان اصلی حالت فهرست:

۱. سامانه داده درخواست را اعتبارسنجی و رابط و نقطه پایانی ثبت شده را پیدا می‌کند.
۲. روش درخواست باید دقیقاً با روش ثبت شده برای نقطه پایانی برابر باشد.
۳. اعتبار زمانی مجوز دسترسی کاربر بررسی می‌شود.
۴. سمت سرور راز ارائه‌دهنده را به درخواست اضافه و مقصد را به ارائه‌دهنده مجاز محدود می‌کند.
۵. درخواست با محدودیت زمان، اندازه و تغییر مسیر اجرا می‌شود.
۶. پاسخ پالایش شده به کاربر برمی‌گردد و رکورد مصرف شامل وضعیت، تأخیر و اندازه پاسخ ذخیره می‌شود.

جریان جایگزین عمومی: مهمان می‌تواند یک نشانی عمومی را مستقیم آزمایش کند. در این حالت راز مدیریت شده سرور تزریق نمی‌شود و رویداد مصرف حساب نیز ثبت نمی‌گردد. با این حال، نشانی‌های محلی و خصوصی، سرآیندهای خطرناک و مقصدهای نامعتبر مسدود می‌شوند.

مسیرهای خطا: نبود رابط یا نقطه پایانی، ناهماهنگی روش، مجوز منقضی، نبود پیکربندی ارائه‌دهنده، پایان زمان انتظار یا خطای سرویس بالادستی با پاسخ کنترل شده گزارش می‌شود؛ درخواست نامعتبر در سابقه مصرف ثبت نمی‌شود.

پس شرط: نتیجه فنی آزمایش نمایش داده می‌شود و فقط اجرای معتبر حالت فهرست در تاریخچه مصرف کاربر قرار می‌گیرد.

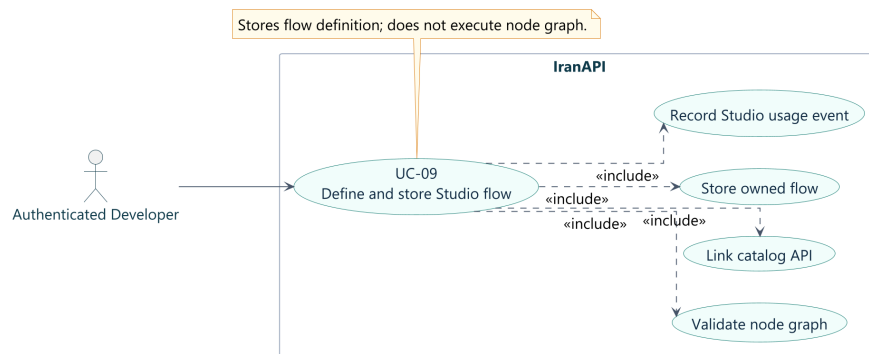
۳-۳-۱۰ مورد کاربرد UC-09: تعریف و ذخیره جریان در محیط طراحی

بازیگر اصلی: توسعه‌دهنده احراز شده.

هدف: تعریف و ذخیره یک جریان گره‌ای مرتبط با رابط.

پیش شرط: کاربر وارد شده و سرویس مرجع در فهرست وجود داشته باشد.

محرک: کاربر نام جریان، سرویس، گره‌ها و ناحیه را برای ذخیره ارسال می‌کند.



شکل ۳-۱۰ نمودار PlantUML مورد کاربرد UC-09

جریان اصلی:

۱. سامانه ساختار ورودی و گره‌های جریان را اعتبارسنجی می‌کند.
۲. سرویس مرجع با شناسه متنی پیدا می‌شود.
۳. جریان با مالکیت کاربر، نام، گره‌ها و ناحیه انتخابی ذخیره می‌شود.
۴. همزمان یک رویداد مصرف با منبع محیط طراحی ثبت می‌گردد.
۵. اطلاعات جریان و مصرف به رابط کاربری برمی‌گردد و جریان در فهرست شخصی کاربر دیده می‌شود؛ در این عملیات گره‌ها اجرا نمی‌شوند.

مسیرهای جایگزین و خطا: کاربر مهمان، سرویس ناموجود یا ساختار گره نامعتبر باعث رد درخواست می‌شود. فهرست جریان‌ها نیز فقط داده‌های متعلق به کاربر جاری را نمایش می‌دهد. **پس شرط:** تعریف جریان قابل بازیابی در فضای کار کاربر ذخیره و رویداد مصرف آن ثبت شده است؛ اجرای واقعی گره‌ها جزو این مورد کاربرد نیست.

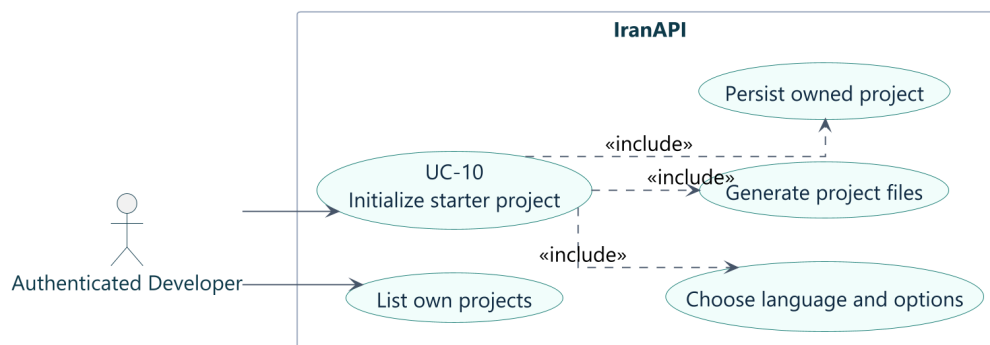
۱۱-۳-۳ مورد کاربرد UC-10: ایجاد پروژه آغازین

بازیگر اصلی: توسعه‌دهنده احراز شده.

هدف: ساخت سریع اسکلت یک پروژه مصرف کننده رابط.

پیش شرط: کاربر نشست معتبر داشته باشد؛ اگر سرویس خاصی انتخاب شده، آن سرویس باید در فهرست موجود باشد.

محرك: کاربر فرمان Init را با نام پروژه و زبان دلخواه ارسال می کند.



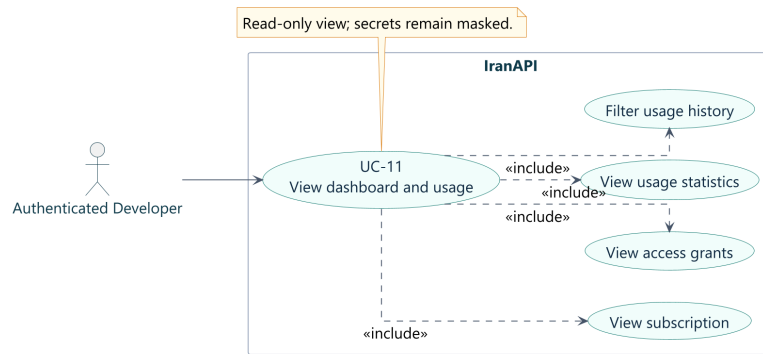
شکل ۱۱-۳ نمودار PlantUML مورد کاربرد UC-10

جریان اصلی:

۱. سامانه زبان های آماده و پروژه های قبلی کاربر را نمایش می دهد.
 ۲. کاربر نام پروژه، زبان، نام بسته، رابط اختیاری و نیاز به Docker را تعیین می کند.
 ۳. سمت سرور ورودی را اعتبارسنجی و قالب متناسب با زبان را انتخاب می کند.
 ۴. ساختار فایل ها و تنظیمات اولیه تولید و به عنوان پروژه متعلق به کاربر ذخیره می شود.
 ۵. مشخصات پروژه تولید شده برای نمایش یا دریافت به رابط کاربری برمی گردد.
- مسیرهای جایگزین و خطا:** برای زبان خارج از فهرست آماده، قالب عمومی ساخته می شود. نام یا داده نامعتبر، سرویس انتخابی ناموجود و درخواست مهمان باعث توقف عملیات می شوند.
- پس شرط:** پروژه آغازین قابل بازیابی با تنظیمات انتخابی در حساب کاربر ثبت شده است.

۳-۳-۱۲ مورد کاربرد UC-11: مشاهده پیشخوان، دسترسی و مصرف

بازیگر اصلی: توسعه‌دهنده احراز شده.
هدف: داشتن نمای یکپارچه از حساب، اشتراک، مجوزهای رابط و الگوی استفاده.
پیش شرط: نشست کاربر معتبر باشد.
محرک: کاربر پیشخوان یا صفحه سابقه مصرف را باز می‌کند.



شکل ۳-۱۲ نمودار PlantUML مورد کاربرد UC-11

جریان اصلی:

۱. رابط کاربری اطلاعات حساب، نمایه، اشتراک جاری و مجوزهای دسترسی را دریافت می‌کند.
 ۲. سامانه شاخص‌های تجمیعی مصرف را برای نمایش در کارت‌های آماری محاسبه می‌کند.
 ۳. سابقه درخواست‌ها با اطلاعات سرویس، منبع رویداد، وضعیت و زمان نمایش داده می‌شود.
 ۴. کاربر می‌تواند سابقه را بر اساس سرویس، منبع یا عبارت جستجو محدود و میان صفحات حرکت کند.
 ۵. اطلاعات حساس مانند کلید کامل در هیچ‌یک از پاسخ‌های پیشخوان نمایش داده نمی‌شود.
- مسیرهای جایگزین و خطا:** برای حساب بدون مصرف یا اشتراک، حالت خالی و مقدارهای پایه نمایش داده می‌شود. نشست منقضی کاربر را از دریافت داده خصوصی بازمی‌دارد و نیاز به ورود دوباره دارد.
- پس شرط:** کاربر وضعیت جاری و سابقه فعالیت خود را می‌بیند؛ مشاهده پیشخوان به‌تنهایی داده عملیاتی را تغییر نمی‌دهد.

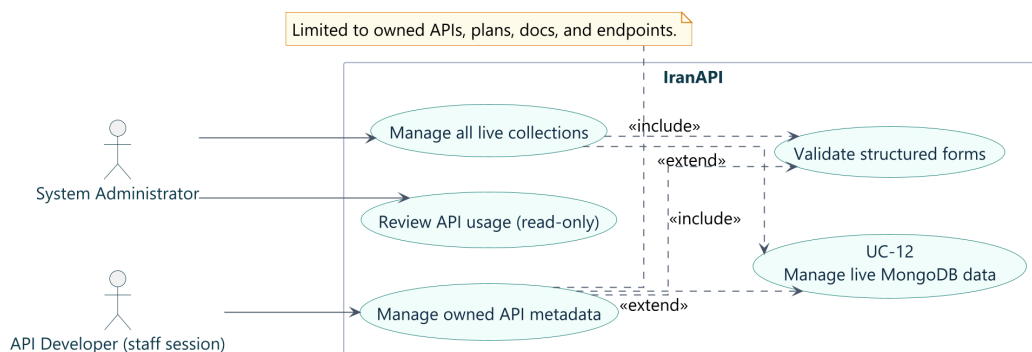
۳-۳-۱۳ مورد کاربرد UC-12: مدیریت داده‌های زنده سامانه

بازیگران اصلی: مدیر سیستم دارای دسترسی ابرکاربر؛ توسعه‌دهنده رابط دارای نشست کارمندی با دامنه محدود.

هدف: نگهداری مجموعه‌های زنده MongoDB از مسیر مدیریتی با کنترل نقش، مالکیت و مجموعه.

پیش‌شرط: نشست معتبر پنل مدیریت وجود داشته باشد؛ مدیر سیستم دسترسی کامل دارد و توسعه‌دهنده رابط فقط به داده متعلق به خود دسترسی می‌گیرد.

محرک: بازیگر یکی از بخش‌های کنسول داده زنده را باز می‌کند.



شکل ۳-۱۳ نمودار PlantUML مورد کاربرد UC-12

جریان اصلی:

۱. پنل بر اساس نقش، فهرست مجموعه‌های مجاز را از داده زنده MongoDB نمایش می‌دهد.
۲. مدیر سیستم می‌تواند همه مجموعه‌های تعریف‌شده را مدیریت کند؛ توسعه‌دهنده رابط فقط رابط‌های متعلق به خود و طرح، مستندات و نقاط پایانی وابسته را می‌بیند.
۳. بازیگر یک رکورد مجاز را ایجاد، مشاهده، ویرایش یا حذف می‌کند.
۴. فرم مدیریتی، فیلدهای ساخت‌یافته و داده‌های تو در تو را پیش از ذخیره اعتبارسنجی می‌کند.
۵. تغییر مستقیماً در مجموعه عملیاتی مربوط ذخیره می‌شود تا نتیجه در فهرست یا بخش حساب قابل مشاهده باشد.
۶. مجموعه مصرف رابط برای مدیر سیستم فقط خواندنی است و امکان افزودن، ویرایش یا حذف ندارد.

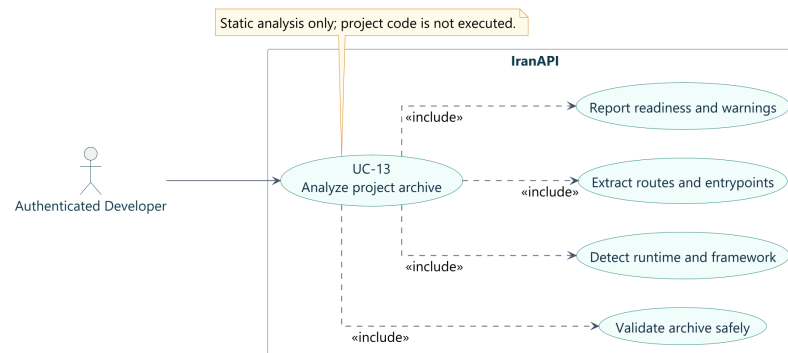
مسیرهای جایگزین و خطا: کاربر عادی و کارمند بدون نقش توسعه‌دهنده رابط نمی‌توانند کنسول زنده را باز کنند. دسترسی توسعه‌دهنده به سند متعلق به کاربر دیگر رد می‌شود. داده نامعتبر ذخیره نمی‌شود و عملیات تغییر روی مجموعه فقط خواندنی پذیرفته نمی‌گردد.

پس‌شرط: داده معتبر در مجموعه زنده ذخیره شده و دامنه مالکیت و سطح دسترسی حفظ شده است.

۳-۳-۱۴ مورد کاربرد UC-13: تحلیل آرشیو پروژه

بازیگر اصلی: توسعه‌دهنده احراز شده.

هدف: شناخت ساختار یک پروژه فشرده پیش از اجرای آن و دریافت برآورد آمادگی استقرار.
پیش شرط: نشست معتبر وجود داشته باشد و فایل ZIP، TAR، TAR.GZ یا TGZ حداکثر ۲۵ مگابایت باشد.
محرك: کاربر آرشیو پروژه را در ابزار تحلیل بارگذاری می‌کند.



شکل ۳-۱۴ نمودار PlantUML مورد کاربرد UC-13

جریان اصلی:

۱. سامانه نوع آرشیو، تعداد فایل و اندازه باز شده را کنترل می‌کند و مسیرهای ناامن، پیوندها و فایل‌های بیش از حد بزرگ را رد می‌کند.
۲. محتوای متنی بدون اجرای کد بررسی می‌شود تا زبان از میان شش زمان اجرای مدیریت شده تشخیص داده شود.
۳. چارچوب، فایل‌های ورودی، فایل‌های مانیفست، Dockerfile، مسیرهای HTTP و پورت احتمالی استخراج می‌شوند.
۴. هش آرشیو و هشدارهایی مانند نبود نقطه ورود، مسیرهای پویا یا وجود فایل رازدار در نتیجه ثبت می‌شود.
۵. سامانه سازنده پیشنهادی، فرمان شروع، زمان اجرای مقصد و مقدار آمادگی استقرار را برمی‌گرداند.

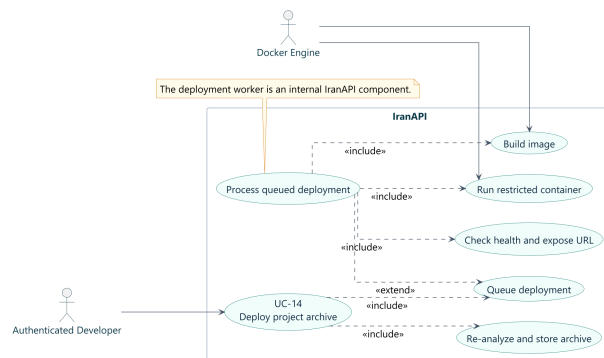
مسیرهای جایگزین و خطا: آرشیو خالی، خراب، بزرگ‌تر از حد، دارای مسیر پیمایش، پیوند نمادین یا پسوند پشتیبانی نشده با خطای اعتبارسنجی رد می‌شود. زبان ناشناخته یا نبود مسیر HTTP هشدار تولید می‌کند، اما در صورت وجود Dockerfile می‌تواند همچنان قابل استقرار باشد.

پس شرط: نتیجه تحلیل بدون اجرای کد در اختیار کاربر است و فایل خام برای این مسیر ذخیره نمی‌شود.

۳-۳-۱۵ مورد کاربرد UC-14: استقرار پروژه فشرده

بازیگر اصلی: توسعه‌دهنده احراز شده.

هدف: پذیرش، ساخت و اجرای یک پروژه رابط در محیط جدا شده و قابل پایش.
پیش شرط: آرشیو تحلیل شده باید زمان اجرای پشتیبانی شده، نقطه ورود یا Dockerfile داشته باشد؛ نام پروژه و ناحیه نیز معتبر باشند.
محرک: کاربر آرشیو، نام پروژه و یکی از نواحی مجاز را برای استقرار ارسال می‌کند.



شکل ۳-۱۵ نمودار PlantUML مورد کاربرد UC-14

جریان اصلی:

۱. سامانه آرشیو را دوباره تحلیل و فقط پروژه آماده را می‌پذیرد؛ سپس فایل را در فضای اختصاصی کاربر با اثرانگشت آن ذخیره می‌کند.
۲. یک رکورد استقرار با وضعیت queued ساخته می‌شود و کارگر استقرار آن را به وضعیت building می‌برد.
۳. کارگر آرشیو را با کنترل مسیر استخراج می‌کند و از Dockerfile موجود یا سازنده زبان محور استفاده می‌کند.
۴. تصویر با زمان ساخت محدود ساخته و محفظه با محدودیت حافظه، پردازنده، فرایند، قابلیت‌های لینوکس و tmpfs اجرا می‌شود.
۵. سلامت محفظه و نگاشت درگاه بررسی می‌شود؛ در موفقیت نشانی عمومی، شناسه محفظه و گزارش ساخت ثبت می‌گردد.
۶. در شکست ساخت یا خروج برنامه، وضعیت failed و دلیل خلاصه ذخیره می‌شود و کاربر می‌تواند جزئیات استقرار را ببیند.

مسیرهای جایگزین و خطا: نام کوتاه یا طولانی، ناحیه نامعتبر، پروژه آماده نشده، آرشیو خارج از فضای اختصاصی، نبود Docker یا پایان زمان ساخت باعث رد یا شکست کنترل شده می‌شود. استقرار هر کاربر فقط با شناسه همان کاربر قابل مشاهده است.
پس شرط: یا استقرار با وضعیت deployed و نشانی قابل بررسی ثبت شده است، یا وضعیت failed همراه دلیل قابل پیگیری باقی می‌ماند.

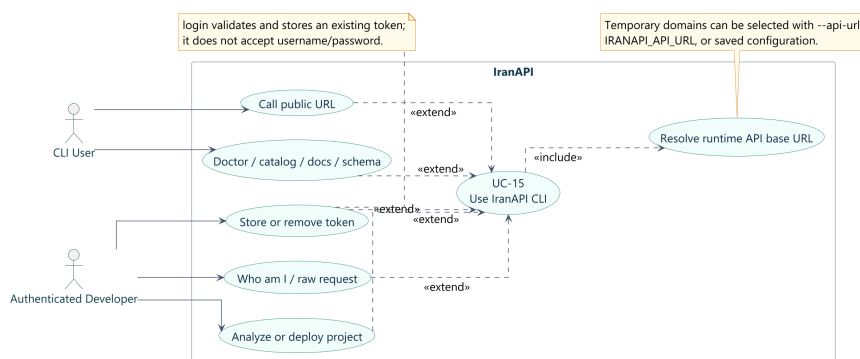
۳-۳-۱۶ مورد کاربرد UC-15: کار با فهرست و پروژه از خط فرمان

بازیگران اصلی: کاربر CLI برای فرمان‌های عمومی؛ توسعه‌دهنده دارای توکن برای فرمان‌های خصوصی.

هدف: خودکارسازی کشف، مستندسازی، فراخوانی عمومی و چرخه پروژه بدون استفاده مستقیم از مرورگر.

پیش‌شرط: نشانی پایه سامانه تنظیم شده باشد؛ فهرست، مستندات، طرح‌واره و ابزار فراخوان عمومی به توکن نیاز ندارند، اما فرمان‌های حساب و پروژه به توکن معتبر نیاز دارند.

محرك: کاربر یکی از فرمان‌های login, doctor, apis, docs, call, analyze یا deploy را اجرا می‌کند.



شکل ۳-۱۶ نمودار PlantUML مورد کاربرد UC-15

جریان اصلی:

1. doctor سلامت سامانه، وضعیت پایگاه داده و اعتبار اختیاری توکن را بررسی می‌کند و توکن را پوشانده نمایش می‌دهد.
 2. کاربر با login توکن را پس از اعتبارسنجی در فایل تنظیمات با مجوز محدود ذخیره می‌کند؛ logout آن را حذف می‌کند.
 3. فرمان‌های apis و docs داده صفحه‌بندی شده فهرست و مستندات را به شکل جدولی یا JSON برمی‌گردانند.
 4. call درخواست عمومی را از مسیر امن ابزار فراخوان عبور می‌دهد و request برای قرارداد خام سامانه استفاده می‌شود.
 5. analyze نتیجه تحلیل ایستای آرشیو و deploy نتیجه صف یا وضعیت نهایی استقرار را برمی‌گردانند؛ گزینه no-wait برای صف‌بندی بدون انتظار وجود دارد.
- مسیرهای جایگزین و خطا:** نبود توکن برای فرمان خصوصی، توکن نامعتبر، نشانی نادرست، پاسخ خطا دار سامانه، فایل بدنه نامعتبر یا پایان زمان انتظار با خطای ساخت یافته و کد مناسب گزارش می‌شود. توکن کامل در خروجی چاپ نمی‌شود.
- پس‌شرط:** نتیجه عملیات در قالب انسانی یا JSON قابل پردازش در اختیار اسکریپت یا توسعه‌دهنده قرار می‌گیرد.

۳-۳-۱۷ روابط و قابلیت ردیابی موارد کاربرد

میان موارد کاربرد وابستگی وجود دارد. UC-02 ادامه طبیعی انتخاب سرویس در UC-01 است و UC-07 به صورت اختیاری از صفحه جزئیات انجام می شود. UC-03 پیش نیاز رفتارهای خصوصی مرورگر از UC-04 تا UC-11 و نیز UC-13 و UC-14 است؛ دسترسی مدیر سیستم در UC-12 از ورود پنل مدیریت و شاخه خصوصی UC-15 از توکن انجام می شود. مجوز فعال پیش از اجرای مدیریت شده ابزار فراخوان در UC-08 کنترل می گردد. اجرای معتبر ابزار فراخوان و ذخیره جریان محیط طراحی رویدادهایی می سازند که در UC-11 دیده می شوند. UC-13 بخشی از تصمیم گیری UC-14 است و UC-15 مسیرهای عمومی و خصوصی را برای خودکارسازی مصرف می کند.

جدول ۳-۵ ردیابی موارد کاربرد به نیازمندی های پروژه

موارد کاربرد	نیازمندی مرتبط	شاهد رفتاری
UC-01، UC-02، UC-07	R1 و R2	فهرست، جزئیات، مستندات، پیشنهاد مشابه و امتیاز
UC-03	R3، R8 و R9	ثبت نام، نشست HttpOnly، خروج و خطای اعتبارسنجی
UC-04، UC-05، UC-11	R4 و R9	نمایه، سازمان، اشتراک، دسترسی، مصرف و کلید پوشانده شده
UC-06	R5 و R8	اعتبارسنجی و انتشار سرویس متعلق به کاربر
UC-08	R6، R8 و R9	اجرای محدود شده درخواست، حفاظت راز و ثبت مصرف
UC-09	R7 و R8	اعتبارسنجی گره ها، ذخیره جریان و ثبت مصرف
UC-10	R4	تولید و نگهداری پروژه آغازین در حساب
UC-12	R8 و R9	کنترل نقش مدیر، دامنه مالکیت توسعه دهنده رابط، فرم معتبر و مجموعه مصرف فقط خواندنی
UC-13	R9 و R11	تحلیل ایستا، محدودیت آرشیو و هشدار فایل های رازدار

موارد کاربرد	نیازمندی مرتبط	شاهد رفتاری
UC-14	R9 و R12	صف ساخت، محفظه محدودشده و جداسازی پروژه کاربر
UC-15	R9 و R13	فرمان‌های عمومی بدون توکن، فرمان‌های خصوصی با توکن، خروجی جدولی/JSON و پوشاندن راز

۳-۴ ورودی‌ها، خروجی‌ها و اعتبارسنجی

ورودی‌های اصلی شامل مشخصات حساب، نمایه، شناسه یا نامک رابط، پالایه‌های جست‌وجو، امتیاز، داده انتشار رابط، شناسه طرح اشتراک، گره‌های محیط طراحی، بایگانی فشرده پروژه، نام پروژه و ناحیه است. در معماری مبتنی بر Django REST Framework، معمولاً تبدیل گر داده همین داده‌ها را کنترل، پاک‌سازی و به شکل قابل استفاده تبدیل می‌کند [۳].

خروجی‌های اصلی سامانه در قالب JSON بازگردانده می‌شوند و برای مصرف سمت کاربر طراحی شده‌اند. پاسخ‌های عمومی معمولاً داده فهرست، مستندات، طرح‌ها و وضعیت سلامت را شامل می‌شوند. پاسخ‌های خصوصی، اطلاعاتی مانند نمایه، سازمان‌ها، کلید پنهان‌شده، اشتراک و تاریخچه مصرف را برمی‌گردانند. جداسازی این دو دسته مهم است، چون داده عمومی برای همه کاربران قابل نمایش است، اما داده خصوصی باید فقط بعد از احراز هویت برگردد.

اعتبارسنجی در این پروژه دو کار اصلی انجام می‌دهد: جلوی ذخیره داده ناقص یا نامعتبر را می‌گیرد و خطا را به پاسخی قابل فهم برای کارخواه تبدیل می‌کند. برای نمونه، هنگام ثبت امتیاز باید مقدار امتیاز معتبر باشد، هنگام انتشار رابط فیلدهای اصلی باید حاضر باشند و هنگام اجرای محیط طراحی ساختار گره‌ها باید قابل پردازش باشد.

۳-۵ امنیت

احراز هویت باید طوری انجام شود که کاربر بعد از ورود به بخش‌های خصوصی برسد، اما داده حساس او در پاسخ‌ها آشکار نشود. در پروژه‌ای که با کلید رابط، توکن و کوکی کار می‌کند، این موضوع فقط یک جزئیات جانبی نیست؛ بخشی از امنیت اصلی محصول است [۲، ۳].

از دید نیازمندی، امنیت فقط ورود کاربر نیست. کنترل دسترسی، نگهداری نشست، جلوگیری از افشای رازها و محدودکردن خروجی‌های حساس هم بخشی از همین موضوع‌اند. آزمون‌های موجود نشان می‌دهند که پنهان‌سازی داده حساس در خروجی‌ها بررسی شده است. تصویر تولیدی رازها را از محیط می‌خواند، حالت اشکال‌زدایی را غیرفعال می‌کند و کوکی امن، CSP، HSTS و شناخت میانجی HTTPS را اعمال می‌کند؛ گام باقی‌مانده ثبت راز ثابت چرخش‌پذیر، محدودسازی دقیق میزبان‌ها و پایش خطاهای عملیاتی است.

۳-۶ جمع‌بندی فصل

نیازهای پروژه حول کشف عمومی رابط، فضای کاری توسعه‌دهنده، مدیریت حساب و اشتراک، و امنیت نشست شکل گرفته‌اند. همین نیازها مبنای فصل معماری هستند؛ چون لایه‌های سامانه باید بتوانند نقش‌ها، جریان‌ها و محدودیت‌های امنیتی را پشتیبانی کنند.

فصل ۴

معماری و طراحی

معماری پروژه را می‌توان از مسیر یک درخواست ساده فهمید. کاربر از رابط React شروع می‌کند، درخواست به سمت سرور Django REST Framework می‌رسد، داده اعتبارسنجی می‌شود و سپس لایه داده نتیجه را آماده می‌کند. توضیح معماری در اینجا بر همین مسیر تکیه دارد، بدون این‌که خواننده را درگیر نام‌های داخلی فایل‌ها کند.

۴-۱ نمای کلی

معماری پروژه سه لایه اصلی دارد: رابط کاربری React، سمت سرور Django REST Framework و لایه داده. این مدل با الگوی رایج برنامه‌های وب نوین هماهنگ است؛ سمت کاربر چیزی را می‌سازد که کاربر می‌بیند، سمت سرور قرارداد رابط برنامه‌نویسی را مدیریت می‌کند و لایه داده مسئول نگهداری اطلاعات محصول است [۹، ۳، ۴].

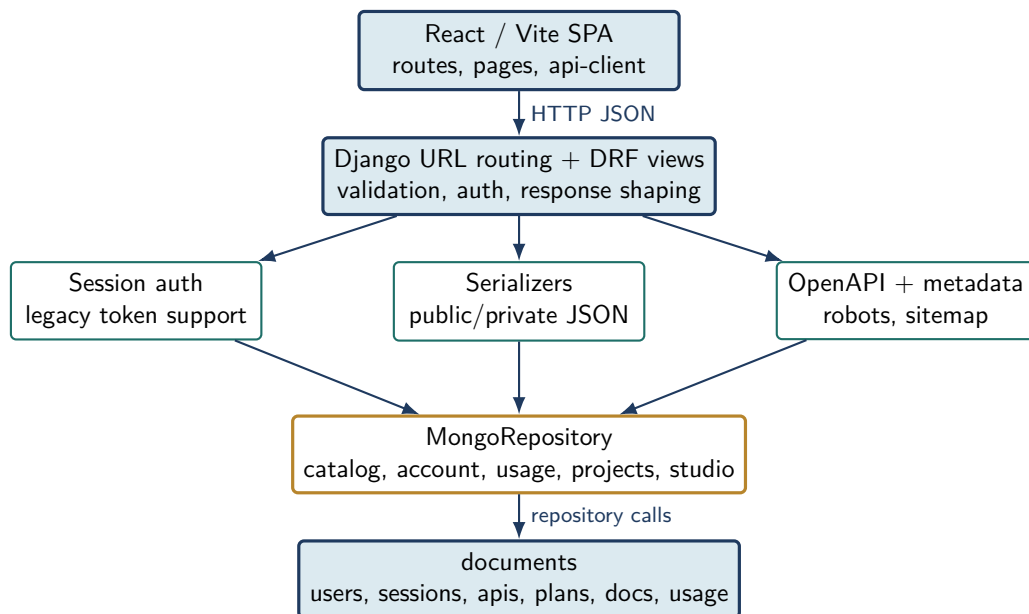
در این طراحی، کاربر نهایی بیشتر با یک برنامه تک‌صفحه‌ای روبه‌رو است، اما عملیات داده از مسیر رابط نسخه‌دار انجام می‌شود. نسخه‌دار بودن مسیرها برای نگهداری آینده اهمیت دارد، چون اجازه می‌دهد قراردادهای جدید اضافه شوند بدون این‌که کارخواه‌های قدیمی ناگهان از کار بیفتند. نمایش پوسته برنامه تک‌صفحه‌ای برای مسیرهای ناشناخته سمت کاربر نیز با مسیریابی کارخواه هماهنگ است.

۴-۲ نمودار معماری

در شکل ۴-۱، مسیر اصلی ارتباط دیده می‌شود: سمت کاربر درخواست‌ها را به رابط سمت سرور می‌فرستد، سمت سرور ورودی را اعتبارسنجی می‌کند، پاسخ را به قالب JSON درمی‌آورد و کار با داده را به لایه مخزن^۱ می‌سپارد.

در این معماری، هر لایه مسئولیت مشخصی دارد. سمت کاربر نمایش، فرم‌ها و حرکت کاربر بین صفحه‌ها را مدیریت می‌کند. سمت سرور احراز هویت، اعتبارسنجی و شکل پاسخ را بر عهده

^۱Repository



شکل ۴-۱ معماری سطح بالای پروژه

دارد. لایه مخزن هم جزئیات ذخیره‌سازی و بازیابی داده را پشت مرزی روشن نگه می‌دارد. این جداسازی کمک می‌کند تغییر در ظاهر کاربر، منطق داده را بی‌دلیل درگیر نکند.

۴-۳ مدل داده

مدل داده پروژه حول چند گروه اصلی می‌چرخد: هویت کاربر، کاتالوگ رابطه‌ها، دسترسی و مصرف، فضای کاری توسعه‌دهنده و اشتراک. این تقسیم‌بندی با مدل سندی MongoDB سازگار است، چون هر گروه می‌تواند ساختار و جزئیات خودش را داشته باشد [۹].

جدول ۴-۱ گروه‌های داده اصلی

گروه	مجموعه‌ها و مسئولیت
هویت	users, sessions و legacy_tokens برای کاربر، نشست و توکن سازگار
کاتالوگ	categories, apis, pricing_plans, documentations و api_endpoints و api_ratings

گروه	مجموعه‌ها و مسئولیت
مصرف	access_grants و api_usage برای مجوز دسترسی و رخدادهای مصرف
فضای کاری	organizations و studio_flows برای امکانات توسعه‌دهنده
پروژه و استقرار	api_projects و project_deployments برای تحلیل، صف ساخت، وضعیت و نشانی اجرای پروژه
اشتراک	subscription_plans، user_subscriptions و subscription_checkouts

گروه هویت پایه دسترسی کاربر را می‌سازد و به نشست‌ها و نشانه‌های سازگار قدیمی متصل است. گروه کاتالوگ محتوای عمومی بازارچه را نگه می‌دارد؛ یعنی دسته‌ها، رابط‌ها، طرح قیمت‌گذاری، مستندات، نقاط پایانی و امتیازها. گروه مصرف و اشتراک داده‌هایی را ثبت می‌کند که برای پیشخوان، کنترل دسترسی و گزارش مصرف لازم‌اند. گروه فضای کاری نیز قابلیت‌های مخصوص توسعه‌دهنده مانند سازمان و جریان طراحی را پشتیبانی می‌کند. گروه پروژه و استقرار، مشخصات پروژه‌های تولیدشده، اثرانگشت آرشیو، تحلیل ایستا، صف ساخت، گزارش اجرا و نشانی سرویس را نگه می‌دارد؛ محتوای آرشیو در فضای اختصاصی کاربر ذخیره می‌شود.

۴-۴ مرزهای خارجی

مرزهای خارجی پروژه شامل ارائه‌دهندگان ورود اجتماعی، سرویس مقصد، موتور Docker و بازارهای بیرونی مانند RapidAPI است [۱۱]. کارخواه خط فرمان نیز مصرف‌کننده بیرونی قرارداد نسخه‌دار به شمار می‌آید. این مرزها باید روشن باشند، چون هر کدام سطحی از ریسک و وابستگی بیرونی وارد سامانه می‌کنند.

شناخت مرز خارجی برای تحلیل ریسک مهم است. ورود اجتماعی به پیکربندی ارائه‌دهنده وابسته است و در صورت نبود auth_url نمی‌تواند کامل اجرا شود. ابزار فراخوان مقصد را با فهرست مجاز و کنترل DNS محدود می‌کند و راز ارائه‌دهنده را در سرور نگه می‌دارد. تحلیل آرشیو بدون اجرای کد انجام می‌شود؛ پس از آن کارگر استقرار با Docker تصویر می‌سازد و محفظه را با محدودیت منابع و قابلیت‌ها اجرا می‌کند. بنابراین گزارش میان قابلیت پیاده‌سازی‌شده و اتصال تولیدی واقعی تفاوت می‌گذارد.

۴-۵ جمع‌بندی فصل

معماری پروژه یک برنامه تک‌صفحه‌ای متصل به رابط سمت سرور است. این طراحی برای نسخه فعلی مناسب است و فضای پروژه را به یک سکوی واقعی رابط‌های برنامه‌نویسی نزدیک می‌کند. برای

استفاده تولیدی، تصمیم نهایی درباره پایگاه داده، مدیریت رازها و اتصال‌های بیرونی باید دقیق‌تر شود.

فصل ۵

پیاده‌سازی

در پیاده‌سازی، مهم‌ترین نکته این است که کاربر از بیرون چه رفتاری می‌بیند و پشت آن رفتار، چه بخش‌هایی با هم کار می‌کنند. به همین دلیل، متن به جای تکیه بر نام فایل‌ها و مسیرهای داخلی، مسیرهای اصلی محصول را توضیح می‌دهد: از کاتالوگ و ورود گرفته تا پیشخوان، انتشار رابط، ابزار فراخوان، محیط طراحی جریان و تحلیل و استقرار پروژه.

۵-۱ مسیرهای سمت سرور

سمت سرور مجموعه‌ای از مسیرهای نسخه‌دار دارد که بخش‌های اصلی محصول را پوشش می‌دهند: سلامت سامانه، قرارداد OpenAPI، ثبت‌نام، ورود، خروج، حساب کاربری، نمایه، کلید دسترسی، سازمان‌ها، اشتراک، مصرف، کاتالوگ، مستندات، نقاط پایانی، ابزار فراخوان، محیط طراحی جریان، تولید پروژه، تحلیل آرشیو و صف استقرار. نسخه‌دار بودن مسیرها کمک می‌کند توسعه آینده کنترل‌شده‌تر باشد و کارخواه‌ها با تغییرات ناگهانی روبه‌رو نشوند [۳، ۱۰].

این مسیرها دو سطح تجربه می‌سازند. سطح اول عمومی است و برای مشاهده کاتالوگ، جزئیات رابط و مستندات استفاده می‌شود. سطح دوم خصوصی است و پس از ورود کاربر فعال می‌شود؛ مانند پیشخوان، مصرف، اشتراک، سازمان و انتشار رابط. همین جداسازی باعث می‌شود کاربر مهمان با داده خصوصی روبه‌رو نشود و توسعه‌دهنده پس از ورود، فضای کاری خود را ببیند.

۵-۲ لایه مخزن

لایه مخزن منطق کار با داده را متمرکز می‌کند. ساخت کاربر، نگهداری نشست، چرخش کلید دسترسی، خواندن کاتالوگ، ثبت مصرف ابزار فراخوان، ذخیره جریان‌های طراحی و مدیریت رکوردهای پروژه و استقرار از همین مسیر انجام می‌شود. این شیوه باعث می‌شود نماها بیش از حد شلوغ نشوند و جزئیات داده در یک لایه مشخص بماند.

در پروژه‌ای شبیه بازارچه رابط‌های برنامه‌نویسی، این جداسازی اهمیت زیادی دارد. داده‌های حساب، کاتالوگ، مستندات، اشتراک و مصرف شکل یکسانی ندارند. وقتی این بخش‌ها پشت یک مخزن قرار می‌گیرند، تغییر در روش ذخیره‌سازی کمتر به بخش‌های دیگر منت می‌کند.

۵-۳ تبدیل گر داده و قرارداد پاسخ

تبدیل گرهای داده مرز امن بین داده داخلی و خروجی قابل نمایش هستند. داده‌ای که داخل سامانه نگهداری می‌شود، همیشه نباید با همان شکل به کاربر برگردد. برای نمونه، مقدار کامل کلید دسترسی نباید در پاسخ عادی دیده شود و بهتر است فقط نسخه پنهان شده آن نمایش داده شود. این رویکرد با نقش تبدیل گر داده در Django REST Framework هماهنگ است [۳].

در بخش انتشار رابط نیز تبدیل گر داده کمک می‌کند داده ورودی پیش از ثبت بررسی شود: نام، نشانی پایه، روش احراز هویت، دسته و توضیح سرویس باید روشن باشند. نتیجه این کار، پاسخ‌هایی است که برای سمت کاربر پیش‌بینی پذیرتر و برای کاربر فهم‌پذیرترند.

۵-۴ سمت کاربر

سمت کاربر همان جایی است که کاربر بیشترین تماس را با محصول دارد: صفحه مرور رابط‌ها، جزئیات سرویس، مستندات، قیمت‌گذاری، پرداخت، ورود، ثبت‌نام، پیشخوان، سازمان‌ها، انتشار، ابزار فراخوان، محیط طراحی جریان، تولید اسکلت پروژه و ابزار تحلیل و استقرار. معماری مؤلفه‌ای React برای چنین تجربه‌ای مناسب است، چون بخش‌های تکرارشونده مانند فرم، کارت، جدول و وضعیت بارگذاری را می‌توان جدا نگه داشت [۴].

ارتباط سمت کاربر با سمت سرور از طریق کارخواه‌های مشخص انجام می‌شود. یک گروه درخواست‌ها برای کاتالوگ است، یک گروه برای احراز هویت و یک گروه برای حساب کاربری و ابزارهای توسعه‌دهنده. این جداسازی باعث می‌شود صفحه‌ها کمتر به جزئیات نشانی‌ها وابسته باشند و نگهداری پروژه ساده‌تر شود.

۵-۵ جریان‌های اصلی پیاده‌سازی

جدول ۵-۱ نشان می‌دهد هر جریان مهم از نگاه کاربر چه معنایی دارد و در پشت صحنه کدام بخش‌ها را درگیر می‌کند.

جدول ۵-۱ جریان‌های اصلی در پیاده‌سازی

جریان	بخش‌های درگیر	خروجی برای کاربر
مرور کاتالوگ	سمت کاربر، کارخواه کاتالوگ، سمت سرور و لایه داده	فهرست قابل جستجو و صفحه‌بندی شده رابط‌ها
جزئیات رابط	صفحه جزئیات، مستندات، طرح‌ها و نقاط پایانی	شناخت سرویس پیش از مصرف
ورود و نشست	فرم ورود، احراز هویت، کوکی نشست و پاسخ کاربر	دسترسی به بخش خصوصی

جریان	بخش‌های درگیر	خروجی برای کاربر
پیشخوان	حساب کاربری، مصرف، اشتراک، سازمان و کلید دسترسی	مدیریت وضعیت توسعه‌دهنده
انتشار رابط	فرم انتشار، اعتبارسنجی و ثبت در کاتالوگ	اضافه‌شدن سرویس جدید به محصول
فراخوان و طراحی جریان	ابزار آزمون، جریان کاری و ثبت مصرف	آزمایش و طراحی جریان‌های مرتبط با رابط
تحلیل و استقرار پروژه	بارگذاری آرشیو، تحلیل ایستا، صف کارگر و اجرای محدودشده	ارزیابی و انتشار پروژه در محیط
کلاینت خط فرمان	احراز هویت، کاتالوگ، مستندات، فراخوانی عمومی و پروژه	مصرف خودکار قرارداد نسخه‌دار

۵-۶ بخش‌های فنی

جدول ۵-۳ نقش بخش‌های فنی را با زبان قابل خواندن خلاصه می‌کند، بدون این‌که گزارش به فهرست نام‌های داخلی پروژه تبدیل شود.

جدول ۵-۳ بخش‌های فنی و نقش آن‌ها در پروژه

بخش	نقش در سامانه	اثر در گزارش
مسیریابی سمت سرور	تعریف سطح رابط و نسخه‌بندی مسیرها	توضیح قرارداد ارتباطی سامانه
نما و تبدیل گر داده	دریافت اعتبارسنجی و پاسخ	توضیح رفتار حساب، کاتالوگ و ابزارها
مخزن داده	مدیریت داده‌های کاربر، کاتالوگ، مصرف و اشتراک	توضیح مدل عملیاتی داده
رابط کاربری	ساخت صفحه‌ها و جریان حرکت کاربر	توضیح تجربه محصول

بخش	نقش در سامانه	اثر در گزارش
کارخواه سمت کاربر	ارتباط کنترل شده با سمت سرور	توضیح اتصال صفحه ها به رابط برنامه نویسی
اجرای محلی	راه اندازی سمت کاربر و سمت سرور کنار هم	توضیح مسیر ارائه و آزمون دستی
آزمون ها	کنترل رفتارهای اصلی و خطاهای مهم	پشتوانه بخش ارزیابی

۵-۷ کلاینت خط فرمان و دامنه پویا

کارخواه خط فرمان برای استفاده تعاملی و خودکار طراحی شده است. نشانی پایه رابط داخل برنامه ثابت نشده و در هر اجرا می تواند با `api-url`، متغیر محیطی `IRANAPI_API_URL` یا پیکربندی ذخیره شده تعیین شود. این تصمیم برای دامنه های موقت استقرار اهمیت دارد و مانع وابستگی ابزار به یک میزبان خاص می شود. خروجی انسانی از رنگ های معنایی، جدول های UTF-8 و نشانگر متحرک استقرار استفاده می کند؛ در مقابل، گزینه `json` خروجی پایدار و بدون تزئین برای برنامه های خودکار می سازد.

۵-۸ تحویل تولیدی، کشف پذیری و کارایی

تصویر تولیدی واحد، سمت کاربر ساخته شده و سمت سرور را روی درگاه سرویس ارائه می کند. در این محیط حالت اشکال زدایی^۱ غیرفعال است، پروتکل امن میانجی^۲ مورد اعتماد قرار می گیرد، سربرگ های `HSTS`، `CSP` و سیاست مجوزها اعمال می شوند و حساب آزمایشی ثابت ساخته نمی شود. اگر راز جنگو صریحاً تنظیم نشده باشد، راه انداز یک راز موقت امن می سازد؛ برای نشست های پایدار، تنظیم راز دائمی در محیط استقرار همچنان لازم است. فایل های اثر انگشت دار سمت کاربر با `gzip` و سیاست تغییرناپذیر^۳ بلندمدت ارائه می شوند. مسیرهای `robots.txt` و `sitemap.xml` نشانی کامل را از دامنه همان درخواست می سازند و نشانی معیار^۴، داده ساختاریافته، پرونده معرفی^۵ و `llms.txt` نیز برای موتور جستجو و عامل های هوشمند فراهم شده اند. نسخه `v21` همین تصویر روی `Runflare` فعال است؛ هیچ دامنه موقتی در کد یا گزارش عملیاتی ثابت نشده و رابط خط فرمان با فرمان `connect` میزبان تازه را بدون نصب یا تغییر پروژه می پذیرد. نمونه های `curl` در صفحات اصلی و جزئیات سرویس نیز از مبدأ همان صفحه

^۱DEBUG

^۲Proxy

^۳Immutable

^۴Canonical URL

^۵Manifest

ساخته می‌شوند، بنابراین ردپای میزبان قبلی به کاربر نشان داده نمی‌شود.

۵-۹ نمونه کد

نمونه زیر بخش کوچکی از منطق پنهان کردن داده حساس را نشان می‌دهد.

```
1 SENSITIVE_KEYS = {"password", "api_key", "token", "secret", "authorization"}
2
3 def redact_secrets(value):
4     if isinstance(value, dict):
5         return {
6             key: "[REDACTED]" if str(key).lower() in
7                 SENSITIVE_KEYS else redact_secrets(item)
8             for key, item in value.items()
9         }
10    return value
```

این قطعه کوتاه است، اما برای چنین پروژه‌ای اهمیت زیادی دارد. سامانه با گذرواژه، کلید دسترسی، نشانه و اطلاعات احراز هویت سروکار دارد. اگر این داده‌ها بدون کنترل وارد پاسخ‌ها یا گزارش‌های رخداد شوند، امنیت کاربر آسیب می‌بیند. جایگزین کردن مقدار حساس با [REDACTED] کمک می‌کند خروجی‌ها برای بررسی فنی قابل استفاده باشند، اما رازهای کاربر آشکار نشوند.

۵-۱۰ جمع‌بندی فصل

پیاده‌سازی پروژه از چند بخش روشن ساخته شده است: سمت کاربر برای تجربه کاربر، سمت سرور برای قرارداد رابط، تبدیل‌گر داده برای کنترل ورودی و خروجی و مخزن برای کار با داده. کنار هم قرار گرفتن این بخش‌ها باعث می‌شود پروژه از یک نمونه ساده فراتر برود و به سکویی واقعی برای اشتراک‌گذاری رابط‌های برنامه‌نویسی نزدیک شود.

فصل ۶

آزمون و ارزیابی

پس از پیاده‌سازی، مهم‌ترین پرسش این است که آیا مسیرهای اصلی سامانه واقعاً پایدار کار می‌کنند یا نه. در این بخش، نتیجه آزمون‌های سمت سرور آمده و مشخص شده بخش‌هایی مانند احراز هویت، کاتالوگ، مصرف، امنیت خروجی و قرارداد عمومی رابط چگونه بررسی شده‌اند.

۶-۱ راهبرد آزمون

آزمون‌های سمت سرور چند بخش را پوشش می‌دهند: آماده‌بودن برنامه، پنهان‌سازی رازها، اعتبارسنجی داده، احراز هویت، کاتالوگ، مستندات، امتیازدهی، انتشار رابط، ابزار فراخوان، محیط طراحی جریان، سازمان‌ها، اشتراک، نمایه، تولید پروژه، تحلیل آرشیو، استقرار جداشده، طرح OpenAPI و مسیرهای فراداده سایت. چارچوب آزمون در Django برای همین نوع کنترل رفتار سمت سرور استفاده می‌شود [۲].

راهبرد آزمون ترکیبی است. بخشی از آزمون‌ها رفتار زیرساختی را بررسی می‌کنند؛ مانند آماده‌بودن برنامه، شاخص‌ها و ترتیب برنامه‌های نصب‌شده. بخشی دیگر به رفتار محصول مربوط است؛ مانند ثبت‌نام، ورود، مشاهده فهرست، انتشار رابط و مدیریت اشتراک. گروه سوم آزمون‌ها روی امنیت و قرارداد خروجی تمرکز دارد؛ مانند پنهان‌سازی رازها، محدودبودن مسیرهای خصوصی و معتبر بودن طرح OpenAPI.

۶-۲ نتایج اجرا

دستور زیر در ریشه پروژه اجرا شد:

```
python manage.py test api
```

موجودی نسخه جاری 109 آزمون سمت سرور است. اجرای کامل آن روی MongoDB جداشده در محیط محلی تکرار شد و هر 109 آزمون موفق بود؛ پایگاه آزمون پس از اجرا حذف شد. پایگاه نسخه مستقر نیز در آزمون زنده وضعیت «فعال» گزارش کرد.

این نتیجه نشان می‌دهد سمت سرور در محدوده آزمون‌های موجود پایدار است. ساخت و بررسی ایستا و چندزبانه سمت کاربر، آزمون کارخواه خط فرمان، اجرای کامل مرورگر و بررسی سرویس مستقر نیز جداگانه انجام شدند. با این حال، آزمون بار طولانی، آزمون نفوذ و اتصال پرداخت واقعی هنوز خارج از دامنه ارزیابی فعلی‌اند.

۳-۶ جدول آزمون‌ها

جدول ۱-۶ خلاصه آزمون‌های اجراشده

شناسه	مؤلفه	سناریو	انتظار	نتیجه واقعی	وضعیت
T1	امنیت	حذف گذرواژه، کلید و توکن از داده	رازها دیده نشوند	موفق	قبول
T2	مدل‌ها	نامک فارسی یکتا و اعتبار رنگ و امتیاز	خطا برای داده نامعتبر	موفق	قبول
T3	احراز هویت	ثبت‌نام، ورود، نشست و خروج	نشست ساخته و پاک شود	موفق	قبول
T4	کاتالوگ	فهرست، جزئیات، مستندات و نقاط پایانی	داده صفحه‌بندی	موفق	قبول
T5	حساب	نمایه، کلید دسترسی، سازمان و دسترسی	مسیر خصوصی با احراز هویت	موفق	قبول
T6	اشتراک	ساخت، مشاهده، لغو و تأیید پرداخت دستی	وضعیت درست پرداخت	موفق	قبول
T7	طراحی فراخوان	و ثبت مصرف پس از فراخوان یا ذخیره جریان طراحی	رکورد مصرف ساخته شود	موفق	قبول
T8	طرح و سایت	OpenAPI، robots.txt و sitemap.xml	پاسخ معتبر	موفق	قبول

شناسه	مؤلفه	سناریو	انتظار	نتیجه واقعی	وضعیت
T9	پروژه	تحلیل آرشیو، مسیر امن، صف استقرار و شکست ساخت	رد ورودی ناامن و ثبت وضعیت	موفق	قبول
T10	سمت کاربر	نوع سنجی، واریسی ایستا، ساخت و سازگاری پنج زبان	بدون خطا	موفق	قبول
T11	کارخواه فرمان	خط فرمان ها، پارامترها، دامنه پویا و خروجی JSON	26/26 آزمون موفق	موفق	قبول
T12	استقرار زنده	سلامت، متادیتا، امنیت، و امتیاز بالا	پاسخ معتبر و امتیاز بالا	موفق	قبول

Lighthouse
Runflare روی

جدول نشان می دهد پوشش آزمون ها به یک قابلیت محدود نیست و بیشتر مسیرهای اصلی محصول را لمس می کند. وجود آزمون برای مسیرهای OpenAPI و متادیتای سایت نیز اهمیت دارد، زیرا این مسیرها مستقیماً به مصرف کنندگان بیرونی و موتورهای جستجو مربوط می شوند. آزمون های امنیتی هم بررسی می کنند داده های حساس در پاسخ عادی سامانه آشکار نشوند.

۴-۶ محدودیت های آزمون

آزمون سمت کاربر با Playwright پس از استقرار نهایی دوباره اجرا شد. هر 48 اجرای محلی، شامل دو آزمون مدیریتی، در رایانه و تلفن همراه موفق بود. در محیط تولید هر 46 اجرای ایمن موفق شد؛ دو اجرای مخرب پنل مدیر ارشد^۱ چون اطلاعات اعتبارسنجی مدیریتی عمداً به محیط تولید داده نشد، به صورت صریح کنار گذاشته شدند. مسیرهای عمومی، ثبت نام، ورود، ابزارهای پروژه، امتیازدهی، خروج، پنل توسعه دهنده و مسیر نامعتبر پوشش داده شدند [۱۲].

مجموعه کارخواه خط فرمان نیز 26/26 آزمون موفق داشت. بسته مستقیماً از دامنه زنده نصب شد؛ پس از انقضای دامنه، همان نصب با فرمان connect به دامنه تازه متصل شد و فرمان doctor سلامت رابط و پایگاه داده را تأیید کرد. نمونه Express با سه مسیر نیز از طریق همین کارخواه تحلیل و مستقر شد.

ارزیابی تأییدی Lighthouse روی دامنه موقت تازه نسخه v21 امتیازهای 96 برای کارایی و 100 برای دسترسی پذیری، رویه های برتر و SEO ثبت کرد. FCP حدود 1.8s، LCP حدود 2.37s.

^۱ Superuser

زمان مسدودسازی کل 71ms و جابه‌جایی تجمعی چیدمان صفر بود. اجرای تأییدی نسخه v20 کارایی 98 ثبت کرده بود؛ این اختلاف نوسان آزمایشگاهی شبکه و میزبان موقت را نشان می‌دهد و سه شاخص کیفی دیگر در هر دو نسخه کامل مانده‌اند.

محدودیت باقی‌مانده این است که آزمون بار، مرورگرهای متعدد و اتصال پولی واقعی به ارائه‌دهندگان انجام نشده است. ابزار فراخوان تا مرز امن اطلاعات اعتبارسنجی واقعی بررسی شد: کنترل مجوز، فهرست مجاز^۲، حل DNS عمومی و خطای نبود اطلاعات اعتبارسنجی تأیید شدند، اما بدون کلیدهای مالک سرویس هیچ پیامک یا تراکنش خارجی ارسال نشد.

۵-۶ جمع‌بندی فصل

مجموعه کامل سمت سرور 109/109، هر 46 اجرای ایمن مرورگر و 26/26 آزمون کارخواه خط فرمان موفق شدند؛ دو اجرای مخرب مدیریتی در محیط تولید آگاهانه کنار گذاشته شدند. ساخت سمت کاربر، پنج بسته ترجمه، سلامت MongoDB، قراردادهای اصلی و نسخه زنده Runflare تأیید شدند.

^۲Allowlist

فصل ۷

نتایج و بحث

در این بخش، پروژه از زاویه نتیجه نهایی بررسی می‌شود: چه چیزی آماده شده، کدام بخش‌ها برای ارائه قابل اتکا هستند و چه کارهایی هنوز باید کامل شوند. هدف، ارائه تصویر واقعی‌تری از وضعیت سامانه است، نه فقط آوردن فهرستی از قابلیت‌ها.

۷-۱ ویژگی‌های پیاده‌سازی شده

ویژگی‌های اصلی پروژه شامل فهرست عمومی، صفحه جزئیات رابط، مستندات، طرح‌ها، ثبت امتیاز، ثبت‌نام، ورود، نشست، خروج، نمایه، چرخش کلید رابط، سازمان‌ها، اشتراک، پرداخت دستی، مصرف، ابزار فراخوان، محیط طراحی جریان، تولید پروژه، تحلیل بایگانی، صف‌بندی استقرار ایزوله و انتشار رابط است. کارخواه خط فرمان نیز همین قرارداد را برای فهرست، مستندات، فراخوانی عمومی و پروژه‌ها مصرف می‌کند. کنار هم قرار گرفتن این بخش‌ها، پروژه را به یک بازارچه رابط برنامه‌نویسی نزدیک می‌کند [۱۱].

این ویژگی‌ها دو نوع ارزش ایجاد می‌کنند. ارزش نخست برای مصرف‌کننده رابط است: کشف، مقایسه، مشاهده مستندات و انتخاب طرح. ارزش دوم برای توسعه‌دهنده یا مالک رابط است: مدیریت حساب، سازمان، کلید، انتشار، مشاهده مصرف و آزمایش جریان‌ها. همین دواگانگی باعث می‌شود پروژه از یک فهرست ساده فراتر برود و به شکل یک بازارچه عملیاتی نزدیک شود.

۷-۲ نمونه خروجی رابط

طرح OpenAPI برای معرفی مسیرها و پاسخ‌ها استفاده می‌شود. نمونه زیر پاسخ ساده مسیر سلامت را نشان می‌دهد [۱۰]:

```
1 GET /api/v1/system/health/
2 {
3   "status": "ok",
4   "database": "up",
5   "timestamp": "2026-05-17T00:00:00Z"
```

این پاسخ کوتاه است، اما برای ارزیابی اهمیت دارد؛ زیرا نشان می‌دهد مسیر نسخه‌دار رابط فعال است و کارخواه می‌تواند وضعیت سرویس را بدون ورود بررسی کند. وجود زمان در پاسخ نیز برای عیب‌یابی و پایش اولیه مفید است.

۷-۳ استقرار و کیفیت نسخه زنده

نسخه v21 سامانه با تصویر تولیدی روی Runflare مستقر شد و یک ابرک فعال از یک ابرک مورد انتظار گزارش شد. پس از انقضای دامنه یک‌ساعته، دامنه تازه ساخته شد و مسیر سلامت، صفحه اصلی، کاتالوگ، robots.txt، sitemap.xml، manifest کلاینت و بسته داندودی از همان میزبان دوباره بررسی شدند. هیچ ردپایی از دامنه منقضی در پاسخ‌های تازه باقی نماند. بهینه‌سازی ارائه فایل‌های ایستا، کارایی نسخه زنده v21 را در دو اجرای تأییدی به امتیاز 96 رساند و سه شاخص دسترس‌پذیری، رویه‌های برتر و بهینه‌سازی برای موتورهای جست‌وجو را به امتیاز کامل 100 رساند. نسخه v20 پیش‌تر امتیاز کارایی 98 ثبت کرده بود. فشرده‌سازی پاسخ، حافظه نهان و فایل‌های دارای درهم‌سازی از اتلاف انتقال و بارگیری دوباره جلوگیری می‌کنند. فایل llms.txt و طرح OpenAPI نیز مسیرهای معتبر و نسخه‌دار را برای عامل‌های نرم‌افزاری معرفی می‌کنند.

۷-۴ نقاط قوت

نقاط قوت پروژه را می‌توان در چند مورد خلاصه کرد: مسیرهای نسخه‌دار، پاسخ خطای یکنواخت، آزمون‌های متعدد، پنهان‌کردن رازها، پشتیبانی از فارسی، قرارداد OpenAPI، اجرای محلی با Docker Compose و استقرار زنده امن با فراداده وابسته به دامنه زمان اجرا. از نظر نگهداری، مهم‌ترین نقطه قوت پروژه جداسدن لایه‌هاست. مسیرها، تبدیل‌گرهای داده، لایه داده و کارخواه سمت کاربر نقش‌های نسبتاً روشن دارند. از نظر تجربه کاربر، پشتیبانی از راست‌به‌چپ و فارسی باعث می‌شود سامانه برای مخاطب ایرانی مناسب‌تر باشد. از نظر توسعه، وجود آزمون‌ها و طرح OpenAPI مسیر ادامه کار و اتصال کارخواه‌های دیگر را ساده‌تر می‌کند.

۷-۵ چالش‌ها و بدهی فنی

مهم‌ترین چالش باقی‌مانده، دوام داده و اتصال سرویس‌های بیرونی واقعی است. تصویر تولیدی اکنون حالت اشکال‌زدایی را غیرفعال و سربرگ‌های امنیتی را اعمال می‌کند، اما راز ثابت جنگو باید در تنظیمات میزبان ثبت شود و پایگاه MongoDB برای بهره‌برداری بلندمدت به دیسک پایدار، پشتیبان‌گیری و آزمون بازیابی نیاز دارد. ورود اجتماعی، پرداخت و ابزار فراخوان نیز برای بهره‌برداری واقعی به اطلاعات اعتبارسنجی ارائه‌دهندگان وابسته‌اند. این چالش‌ها مانع نمایش نسخه زنده نیستند، اما برای بهره‌برداری عمومی باید حل شوند. پرداخت دستی و اجرای نمونه ابزار فراخوان باید با سرویس‌های واقعی، کنترل خطا، محدودیت

زمانی و ثبت رویداد دقیق تکمیل شوند. پایش، هشدار و سیاست بازیابی نیز باید بیرون از فرایند برنامه تعریف شوند.

۶-۷ بحث

پیاده‌سازی فعلی یک نمونه زنده و قابل آزمون از بازارچه رابط برنامه‌نویسی است. با این حال، برای استفاده تولیدی پایدار باید رازهای ثابت میزبان، پایش و پشتیبان‌گیری MongoDB، اجرای آزمون‌ها در یکپارچه‌سازی پیوسته و مسیر پرداخت واقعی دقیق‌تر شوند.

سطح بلوغ پروژه را می‌توان «نمونه کامل و مستقر برای ارزیابی» دانست. قابلیت‌های اصلی محصول وجود دارند و آزمون‌های بک‌اند، مرورگر، خط فرمان و Lighthouse از آن‌ها پشتیبانی می‌کنند، اما همه نیازهای یک سامانه تولیدی هنوز کامل نشده‌اند؛ از جمله دوام داده، اتصال‌های بیرونی واقعی، مشاهده‌پذیری و پایداری زیر بار.

۷-۷ جمع‌بندی فصل

نتیجه پروژه یک درگاه نسبتاً کامل، مستقر و قابل بررسی برای کشف، انتشار و مدیریت رابط‌هاست. کیفیت فنی و تجربه وب در آزمون‌های محلی و زنده وضعیت خوبی دارند؛ دوام عملیاتی و اتصال‌های مالی واقعی مبنای پیشنهاد‌های فصل پایانی‌اند.

فصل ۸

نتیجه‌گیری و کارهای آینده

در پایان گزارش، باید روشن باشد که پروژه چه ارزشی ایجاد کرده و چه فاصله‌ای تا یک محصول آماده بهره‌برداری دارد. این بخش فصل‌های قبل را تکرار نمی‌کند؛ بلکه جمع‌بندی می‌کند سامانه امروز در چه وضعیتی قرار دارد و ادامه کار بهتر است از کجا شروع شود.

۸-۱ جمع‌بندی کار انجام‌شده

IranAPI Hub یک سکوی اشتراک‌گذاری رابط برنامه‌نویسی با سمت سرور Django REST Framework و سمت کاربر React است. سامانه فهرست، مستندات، احراز هویت، پیشخوان، مصرف، اشتراک، سازمان، انتشار، ابزار فراخوان، محیط طراحی جریان، چرخه تحلیل و استقرار پروژه و کارخواه خط فرمان را پوشش می‌دهد.

در طول گزارش، پروژه از دو زاویه بررسی شد: تجربه کاربر و ساختار فنی. فصل نیازمندی‌ها نقش‌ها و جریان‌ها را مشخص کرد. فصل معماری نشان داد این جریان‌ها چگونه بین سمت کاربر، سمت سرور، لایه داده و کارگر استقرار تقسیم می‌شوند. فصل آزمون موفقیت 109/109 آزمون سمت سرور، 48/48 اجرای محلی مرورگر، 46 اجرای ایمن روی نسخه زنده با دو مورد صرف‌نظر مدیریتی کنترل‌شده و 26/26 آزمون کارخواه خط فرمان را ثبت کرد.

۸-۲ دستاوردهای اصلی

دستاوردهای اصلی پروژه شامل پشتیبانی از زبان فارسی، مسیرهای عمومی و خصوصی منظم، خروجی OpenAPI، آزمون‌های چندلایه، ساختار Docker Compose، کلاینت خط فرمان پویا و استقرار زنده Runflare است. نسخه زنده v21 در دو اجرای تأییدی Lighthouse امتیاز 96 کارایی و امتیاز کامل دسترس‌پذیری، رویه‌های برتر و SEO به‌دست آورد؛ نسخه v20 نیز پیش‌تر امتیاز کارایی 98 ثبت کرده بود.

دستاوردهای مهم دیگر، هماهنگی نسبی بین تجربه کاربر و معماری فنی است. کاربر فهرست، مستندات، حساب و ابزارهای توسعه‌دهنده را می‌بیند و پشت آن، رابط، تبدیل‌گر داده، لایه داده و

آزمون قرار گرفته‌اند. همین رابطه باعث می‌شود پروژه فقط نمایشی نباشد و از نظر فنی هم قابل ادامه‌دادن باشد.

۸-۳ محدودیت‌های فعلی

محدودیت‌های فعلی شامل نبود درگاه پرداخت خودکار، نبود کلیدهای واقعی ارائه‌دهندگان برای ارسال درخواست خارجی، نیاز ورود اجتماعی به پیکربندی ارائه‌دهنده، نیاز پایگاه داده پایدار و وابستگی استقرار پروژه‌های کاربران به موتور Docker است. اجرای امن ابزار فراخوان تا کنترل مجوز و اطلاعات اعتبارسنجی واقعی آزمایش شده و تحلیل بایگانی نیز پیش از اجرای کد انجام می‌شود.

این محدودیت‌ها طبیعی‌اند، اما باید در برنامه ادامه کار جدی گرفته شوند. نسخه فعلی برای نمایش و ارزیابی عمومی روی میزبان ابری مناسب است؛ تا زمانی که راز ثابت، پشتیبان‌گیری، پرداخت و اتصال بیرونی دارای اطلاعات اعتبارسنجی کامل نشده باشند، نباید آن را محصول تولیدی آماده داده‌های حساس دانست.

۸-۴ کارهای آینده

کارهای آینده پیشنهادی:

- ثبت راز ثابت و چرخش‌پذیر در محیط میزبان و نگهداری حالت اشکال‌زدایی در وضعیت غیرفعال.
 - پایش و پشتیبان‌گیری عملیاتی MongoDB و آزمون بازیابی داده.
 - اجرای منظم آزمون‌های سمت سرور، Playwright، خط فرمان و Lighthouse در فرایند یکپارچه‌سازی پیوسته.
 - اتصال ابزار فراخوان به سرویس‌های مقصد با کنترل امنیت، محدودیت زمانی و ثبت خطای واقعی.
 - تکمیل پرداخت واقعی، قلاب‌های وب و کنترل سهمیه مصرف.
 - افزودن مستندات نصب تولید، نسخه‌گذاری رابط و پایش عملیاتی.
- ترتیب انجام این کارها بهتر است از ریسک‌های بنیادی شروع شود. ابتدا باید امنیت پیکربندی و رازها سخت‌سازی شود، سپس پشتیبان‌گیری MongoDB و مشاهده‌پذیری عملیاتی اضافه شوند. پس از آن، پرداخت واقعی و اجرای کنترل‌شده ابزار فراخوان با اطلاعات اعتبارسنجی مالک سرویس می‌توانند کامل شوند.

۸-۵ جمع‌بندی فصل

پروژه پایه فنی مناسبی برای یک بازارچه رابط برنامه‌نویسی دارد، روی میزبان ابری فعال است و مجموعه آزمون‌های سمت سرور، سمت کاربر و خط فرمان آن با موفقیت اجرا شده‌اند. مسیر توسعه بعدی باید روی دوام داده، رازهای عملیاتی، یکپارچه‌سازی سرویس‌های بیرونی و پایش متمرکز شود.

با انجام این گام‌ها، سامانه می‌تواند از نمونه زنده کامل به محصولی پایدارتر و آماده‌تر برای استفاده واقعی تبدیل شود.

منابع

- [1] Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/>.
- [2] Django Software Foundation. Django Documentation. <https://docs.djangoproject.com/>.
- [3] Django REST Framework. Official Documentation. <https://www.django-rest-framework.org/>.
- [4] Meta Open Source. React Documentation. <https://react.dev/>.
- [5] Microsoft. TypeScript Handbook. <https://www.typescriptlang.org/docs/>.
- [6] Vite. Vite Guide. <https://vite.dev/guide/>.
- [7] OpenJS Foundation. Node.js Documentation. <https://nodejs.org/docs/latest/api/>.
- [8] Docker. Docker Compose Documentation. <https://docs.docker.com/compose/>.
- [9] MongoDB. MongoDB Manual. <https://www.mongodb.com/docs/manual/>.
- [10] OpenAPI Initiative. OpenAPI Specification. <https://spec.openapis.org/oas/latest.html>.
- [11] RapidAPI. RapidAPI Docs. <https://docs.rapidapi.com/>.
- [12] Microsoft. Playwright Documentation. <https://playwright.dev/docs/intro>.

واژه‌نامه

واژه فارسی	برابر انگلیسی
رابط برنامه‌نویسی کاربردی	Application Programming Interface (API)
سکوی اشتراک گذاری رابط	API Marketplace
سمت سرور	Backend
سمت کاربر	Frontend
پیشخوان	Dashboard
نقطه پایانی	Endpoint
کارخواه	Client
تبدیل گر داده	Serializer
احراز هویت نشست محور	Session Authentication
نشانه سازگار قدیمی	Legacy Token
جریان طراحی	Studio Flow
اطلاعات اعتبارسنجی	Credential
محفظه	Container
مجوز دسترسی	Access Grant

پیوست الف

راهنمای اجرا و شواهد تکمیلی

۱-الف اجرای محلی

اجرای محلی پروژه با Docker Compose انجام می‌شود:

```
❯ docker compose up --build
```

در این پیکربندی، سمت کاربر روی `http://localhost:5173` و سمت سرور روی `http://localhost:8000/api` منتشر می‌شود.

این روش برای بررسی سریع کل سامانه مناسب است، زیرا هر دو بخش اصلی را همزمان بالا می‌آورد. پس از اجرا، کاربر می‌تواند ابتدا مسیر سلامت سمت سرور را بررسی کند و سپس از طریق سمت کاربر وارد جریان مرور فهرست، ورود، پیشخوان و ابزارهای توسعه‌دهنده شود. اگر یکی از سرویس‌ها اجرا نشود، باید گزارش همان سرویس در Docker Compose بررسی شود.

۲-الف اجرای آزمون سمت سرور

دستور اجراشده در این گزارش:

```
❯ python manage.py test api
```

در نسخه فعلی، مجموعه آزمون شامل 109 آزمون سمت سرور است. اجرای آن روی پایگاه ایزوله انجام می‌شود؛ پایگاه روی MongoDB ساخته و پس از آزمون حذف می‌شود. آخرین اجرای کامل نسخه جاری 109/109 آزمون را با موفقیت گذراند.

این دستور فقط آزمون‌های برنامه `api` را اجرا می‌کند و برای تکرار نتیجه فصل آزمون کافی است. در صورت تغییر کد سمت سرور، اجرای دوباره این دستور باید نخستین کنترل کیفی باشد. برای تغییرات سمت کاربر، اجرای آزمون‌های مرورگر و ساخت نسخه سمت کاربر نیز باید به این کنترل اضافه شود.

۳-الف کنترل کامل و استقرار زنده

دستورهای تکمیلی زیر کیفیت سمت کاربر، کارخواه و مرورگر را بررسی می کنند:

```
1 | cd api-hub-express
2 | npm run check:i18n
3 | npm run lint
4 | npx tsc -b --pretty false
5 | npm run build
6 | npx playwright test
7 |
8 | cd ../iranapi-cli
9 | npm test
```

نسخه v21 روی Runflare فعال است. دامنه موقت یک ساعته عمداً در گزارش ثابت نشده است؛ کارخواه نشانی جاری را از `/cli/manifest.json` همان میزبان دریافت و ذخیره می کند:

```
1 | iranapi doctor --api-url https://<current-domain>/api/v1 --
   | json
2 | iranapi apis list --api-url https://<current-domain>/api/v1 --
   | limit 2 --json
```

۴-الف مسیرهای مهم

جدول الف-۱ مسیرهای مهم برای بررسی دستی سامانه

مسیر	کاربرد
/api/v1/system/health/	سلامت سامانه
/api/v1/schema/openapi.json	طرح OpenAPI
/api/v1/auth/login/	ورود نشست محور
/api/v1/catalog/apis/	فهرست و انتشار API
/api/v1/account/usage/	تاریخچه مصرف
/api/v1/account/studio/flows/	جریان های Studio
/api/v1/account/projects/analyze/	تحلیل ایستای آرشیو پروژه
/api/v1/account/projects/deployments/	صف بندی و فهرست استقرار پروژه
/api/v1/account/projects/deployments/<slug>/	جزئیات وضعیت، نشانی و گزارش استقرار